

# **MbedTEE Software Architecture**

**V1.1**



# Revision History

---

| Date       | Reference Code | Summary of Changes |
|------------|----------------|--------------------|
| 06/15/2019 | V1.0           | First Release      |
| 12/28/2022 | V1.1           | Add RISC-V         |

# Notice

---

## **I License**

---

SPDX-License-Identifier: Apache-2.0

## **II Copyright**

---

Copyright © 2019 Xing Loong. All Rights Reserved.

# Abbreviation and Acronyms

---

|             |                                    |
|-------------|------------------------------------|
| <b>ACL</b>  | Access Control List                |
| <b>API</b>  | Application Programming Interface  |
| <b>ASID</b> | Address Space ID                   |
| <b>ASLR</b> | Address Space Layout Randomization |
| <b>DMA</b>  | Direct Memory Access               |
| <b>DRAM</b> | Dynamic Random Access Memory       |
| <b>DSO</b>  | Dynamic Shared Object              |
| <b>ELF</b>  | Executable and Linkable Format     |
| <b>FDT</b>  | Flattened Device Tree              |
| <b>EOF</b>  | End of File                        |
| <b>FIFO</b> | First In First Out                 |
| <b>FD</b>   | File Descriptor                    |
| <b>FAT</b>  | File Allocation Table              |
| <b>FEK</b>  | File Encryption Key                |
| <b>FIQ</b>  | Fast Interrupt Request             |
| <b>FS</b>   | File system                        |
| <b>GIC</b>  | Generic Interrupt Controller       |
| <b>GP</b>   | Global Platform                    |
| <b>DRBG</b> | Deterministic Random Bit Generator |
| <b>HKDF</b> | HMAC-based Key Derivation Function |
| <b>HUK</b>  | Hardware Unique Key                |
| <b>ID</b>   | Identifier                         |
| <b>IPC</b>  | Inter Process Communication        |
| <b>IPI</b>  | Inter Processor Interrupt          |
| <b>IRQ</b>  | Interrupt Request                  |
| <b>ISR</b>  | Interrupt Service Routine          |
| <b>IO</b>   | Input Output                       |

|              |                                                     |
|--------------|-----------------------------------------------------|
| <b>IP</b>    | Intellectual Property                               |
| <b>MMU</b>   | Memory Management Unit                              |
| <b>MIPS</b>  | Microprocessor without Interlocked Pipelined Stages |
| <b>OS</b>    | Operating System                                    |
| <b>OTP</b>   | One Time Programmable                               |
| <b>POSIX</b> | Portable Operating System Interface                 |
| <b>REE</b>   | Rich Execution Environment                          |
| <b>RPC</b>   | Remote Procedure Call                               |
| <b>RPMB</b>  | Replay Protected Memory Block                       |
| <b>SMC</b>   | Secure Monitor Call                                 |
| <b>SGI</b>   | Software Generated Interrupt                        |
| <b>SMP</b>   | Symmetric Multi-Processing                          |
| <b>TA</b>    | Trusted Application                                 |
| <b>TEE</b>   | Trusted Execution Environment                       |
| <b>TLB</b>   | Translation Lookaside Buffer                        |
| <b>TRNG</b>  | True Random Number Generator                        |
| <b>AEAD</b>  | Authenticated Encryption with Associated Data       |
| <b>UART</b>  | Universal Asynchronous Receiver Transmitter         |

# Preface

---

## I Content of the document

---

- This document explains the software architecture behind MbedTEE software, including:
  - ✧ Introduction of the basic concept of MbedTEE operating system.
  - ✧ Introduction of the MbedTEE kernel components.
  - ✧ Introduction of the MbedTEE kernel security.

## II References from the document

---

- [1]: “DDI0406C\_d\_armv7ar\_arm.pdf”
- [2]: “DDI0487G\_a\_armv8\_arm.pdf”
- [3]: “riscv-volume2-privileged-20211203.pdf”
- [4]: “GPD\_TEE\_SystemArch\_v1.2\_PublicRelease.pdf”
- [5]: “GPD\_TEE\_Internal\_Core\_API\_Specification\_v1.2.1\_CC.pdf”
- [6]: “GPD\_TEE\_Client\_API\_v1.0\_EP\_v2.0.pdf”

# Contents

---

|                                                |     |
|------------------------------------------------|-----|
| REVISION HISTORY .....                         | I   |
| NOTICE.....                                    | II  |
| I    LICENSE.....                              | II  |
| II   COPYRIGHT.....                            | II  |
| ABBREVIATION AND ACRONYMS .....                | III |
| PREFACE .....                                  | V   |
| I    CONTENT OF THE DOCUMENT .....             | V   |
| II   REFERENCES FROM THE DOCUMENT .....        | V   |
| CONTENTS .....                                 | VI  |
| LIST OF FIGURES .....                          | IX  |
| LIST OF TABLES .....                           | XI  |
| CHAPTER 1   INTRODUCTION.....                  | 13  |
| 1.1   WHAT IS MBEDTEE.....                     | 14  |
| 1.2   WHAT IS TA .....                         | 14  |
| 1.3   HARDWARE ARCHITECTURE.....               | 14  |
| 1.3.1   ARM TrustZone based architecture ..... | 15  |
| 1.3.2   RISCV based architecture.....          | 17  |
| 1.3.3   MIPS based architecture.....           | 19  |
| CHAPTER 2   OVERVIEW .....                     | 20  |
| 2.1   WORKING MODEL .....                      | 21  |
| 2.2   PROCESSOR INITIALIZATION .....           | 22  |
| 2.3   EXCEPTION LEVELS .....                   | 24  |
| 2.3.1   Exception Levels of ARM .....          | 24  |
| 2.3.2   Exception Levels of RISCV.....         | 25  |
| 2.3.3   Exception Levels of MIPS .....         | 25  |
| 2.4   MULTI-THREADING.....                     | 27  |
| 2.4.1   Thread.....                            | 27  |
| 2.4.2   Process.....                           | 27  |
| 2.4.3   User thread .....                      | 27  |
| 2.4.4   Kernel thread .....                    | 27  |
| 2.4.5   Thread ID.....                         | 28  |
| 2.4.6   POSIX Thread ID.....                   | 28  |
| 2.4.7   Process ID.....                        | 28  |
| 2.5   SCHEDULER .....                          | 29  |
| 2.5.1   SCHED_FIFO.....                        | 29  |
| 2.5.2   SCHED_RR.....                          | 29  |
| 2.5.3   SCHED_OTHER.....                       | 30  |
| 2.5.4   Priority.....                          | 30  |
| 2.6   EXCEPTION HANDLING .....                 | 31  |



|                  |                                                           |           |
|------------------|-----------------------------------------------------------|-----------|
| <b>CHAPTER 3</b> | <b>CONTEXT SWITCH .....</b>                               | <b>32</b> |
| 3.1              | CONTEXT SWITCH OF ARM.....                                | 33        |
| 3.1.1            | REE SMC.....                                              | 33        |
| 3.1.2            | TEE SMC.....                                              | 35        |
| 3.1.3            | AArch32 SVC.....                                          | 35        |
| 3.1.4            | AArch64 SVC.....                                          | 36        |
| 3.1.5            | AArch32 Interrupt.....                                    | 37        |
| 3.1.6            | AArch64 Interrupt.....                                    | 41        |
| 3.2              | CONTEXT SWITCH OF RISCV .....                             | 44        |
| 3.2.1            | System Call .....                                         | 44        |
| 3.2.2            | Interrupt.....                                            | 45        |
| 3.3              | CONTEXT SWITCH OF MIPS .....                              | 46        |
| 3.3.1            | System Call .....                                         | 46        |
| 3.3.2            | Interrupt.....                                            | 47        |
| <b>CHAPTER 4</b> | <b>MBEDTEE COMPONENTS.....</b>                            | <b>48</b> |
| 4.1              | OVERVIEW.....                                             | 49        |
| 4.2              | SYSTEM CALL.....                                          | 49        |
| 4.3              | HEAP MANAGER.....                                         | 50        |
| 4.4              | PAGE MANAGER.....                                         | 50        |
| 4.5              | TIMER FRAMEWORK .....                                     | 50        |
| 4.6              | SYNCHRONIZATION PRIMITIVES .....                          | 51        |
| 4.7              | TASKLET.....                                              | 51        |
| 4.8              | WORKQUEUE.....                                            | 51        |
| 4.9              | IPI .....                                                 | 52        |
| 4.10             | RPC.....                                                  | 52        |
| 4.11             | IPC.....                                                  | 52        |
| 4.11.1           | Signal.....                                               | 52        |
| 4.11.2           | Message Queue/Shared Memory/Pipe/Eventfd/Semaphore.....   | 53        |
| 4.11.3           | Process Spawn and Reaping.....                            | 53        |
| 4.11.4           | User-Space Lock (ulock).....                              | 54        |
| 4.12             | ELF LOADER.....                                           | 54        |
| 4.13             | MMU .....                                                 | 55        |
| 4.14             | FILE SYSTEM .....                                         | 55        |
| 4.14.1           | FS types.....                                             | 55        |
| 4.14.2           | FS Operations.....                                        | 55        |
| 4.15             | STORAGE.....                                              | 56        |
| 4.15.1           | Transient.....                                            | 56        |
| 4.15.2           | Persistent .....                                          | 56        |
| 4.16             | DTB .....                                                 | 57        |
| 4.17             | GLOBALPLATFORM .....                                      | 57        |
| 4.18             | MBEDCRYPTO .....                                          | 57        |
| 4.18.1           | Typical In-Kernel Usage.....                              | 57        |
| 4.18.2           | Symmetric Ciphers.....                                    | 58        |
| 4.18.3           | AEAD (Authenticated Encryption with Associated Data)..... | 58        |
| 4.18.4           | Hash / Message Digest .....                               | 58        |
| 4.18.5           | Message Authentication Code (MAC).....                    | 58        |
| 4.18.6           | Key Derivation Functions (KDF) .....                      | 58        |

|                                     |                                                     |           |
|-------------------------------------|-----------------------------------------------------|-----------|
| 4.18.7                              | <i>Asymmetric Key Exchange and Encryption</i> ..... | 58        |
| 4.18.8                              | <i>Digital Signatures</i> .....                     | 58        |
| 4.18.9                              | <i>Utilities / Big-Number</i> .....                 | 59        |
| <b>CHAPTER 5 TEE SECURITY</b> ..... |                                                     | <b>60</b> |
| 5.1                                 | ISOLATION .....                                     | 61        |
| 5.1.1                               | <i>REE-TEE Isolation</i> .....                      | 61        |
| 5.1.2                               | <i>User-Kernel Isolation</i> .....                  | 61        |
| 5.1.3                               | <i>TA Isolation</i> .....                           | 65        |
| 5.2                                 | ELF MAPPING .....                                   | 66        |
| 5.3                                 | TIMER .....                                         | 68        |
| 5.3.1                               | <i>Overview</i> .....                               | 68        |
| 5.3.2                               | <i>Monotonic Counter</i> .....                      | 68        |
| 5.3.3                               | <i>Time Category</i> .....                          | 68        |
| 5.4                                 | DEBUGGING .....                                     | 69        |
| 5.5                                 | ACCESS CONTROL POLICY .....                         | 70        |
| 5.6                                 | IPC SECURITY .....                                  | 70        |
| 5.6.1                               | <i>Message Queue</i> .....                          | 70        |
| 5.6.2                               | <i>File descriptor sharing</i> .....                | 71        |
| 5.7                                 | RPC SECURITY .....                                  | 71        |
| 5.8                                 | IMAGE SECURITY .....                                | 71        |

# List of figures

---

|                                                              |    |
|--------------------------------------------------------------|----|
| FIGURE 1-1 ARM TRUSTZONE BASED ARCHITECTURE .....            | 15 |
| FIGURE 1-2 FVP_VE_CORTEX_A15x4 PROCESSORS .....              | 16 |
| FIGURE 1-3 RISCv BASED ARCHITECTURE.....                     | 17 |
| FIGURE 1-4 QEMU RISCv PROCESSORS .....                       | 18 |
| FIGURE 1-5 MIPS32 BASED ARCHITECTURE .....                   | 19 |
| FIGURE 2-1 CLIENT IS FROM REE .....                          | 21 |
| FIGURE 2-2 CLIENT IS ANOTHER TA.....                         | 21 |
| FIGURE 2-3 PROCESSOR 0 INITIALIZATION FLOW .....             | 22 |
| FIGURE 2-4 SECONDARY PROCESSOR INITIALIZATION FLOW.....      | 23 |
| FIGURE 2-5 EXCEPTION LEVELS OF AARCH32@ARMv7-A.....          | 24 |
| FIGURE 2-6 EXCEPTION LEVELS OF AARCH64@ARMv8-A/ARMv9-A.....  | 25 |
| FIGURE 2-7 EXCEPTION LEVELS OF RISCv.....                    | 25 |
| FIGURE 2-8 EXCEPTION LEVELS OF MIPS .....                    | 26 |
| FIGURE 3-1 SYNCHRONOUS SMC CALL – AARCH32.....               | 33 |
| FIGURE 3-2 SYNCHRONOUS SMC CALL – AARCH64.....               | 34 |
| FIGURE 3-3 ASYNCHRONOUS SMC CALL – AARCH32.....              | 34 |
| FIGURE 3-4 ASYNCHRONOUS SMC CALL – AARCH64.....              | 35 |
| FIGURE 3-5 SYSTEM CALL – AARCH32 .....                       | 36 |
| FIGURE 3-6 SYSTEM CALL – AARCH64 .....                       | 37 |
| FIGURE 3-7 FIQ INTERRUPTS THE REE - AARCH32.....             | 38 |
| FIGURE 3-8 FIQ INTERRUPTS THE REE (SCHEDULED) - AARCH32..... | 38 |
| FIGURE 3-9 FIQ - AARCH32.....                                | 39 |
| FIGURE 3-10 MBEDTEE FORWARDS IRQ TO REE - AARCH32.....       | 40 |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| FIGURE 3-11 TEE IRQ INTERRUPTS THE REE - AARCH64.....             | 41 |
| FIGURE 3-12 TEE IRQ INTERRUPTS THE REE (SCHEDULED) - AARCH64..... | 42 |
| FIGURE 3-13 REE IRQ INTERRUPTS THE TEE - AARCH64.....             | 42 |
| FIGURE 3-14 TEE IRQ - AARCH64.....                                | 43 |
| FIGURE 3-15 SYSTEM CALL - RISC-V.....                             | 44 |
| FIGURE 3-16 INTERRUPT – RISC-V.....                               | 45 |
| FIGURE 3-17 SYSTEM CALL - MIPS.....                               | 46 |
| FIGURE 3-18 INTERRUPT - MIPS.....                                 | 47 |
| FIGURE 4-1 MBEDTEE COMPONENTS .....                               | 49 |
| FIGURE 4-2 PERSISTENT STORAGE BASED ON REEFS.....                 | 57 |
| FIGURE 5-1 AARCH32 ADDRESS SPACE.....                             | 62 |
| FIGURE 5-2 AARCH64 ADDRESS SPACE.....                             | 63 |
| FIGURE 5-3 RISC-V32 SV32 ADDRESS SPACE .....                      | 64 |
| FIGURE 5-4 RISC-V64 SV39 ADDRESS SPACE .....                      | 64 |
| FIGURE 5-5 MIPS32 ADDRESS SPACE.....                              | 64 |
| FIGURE 5-6 PS INFORMATION .....                                   | 69 |

# List of tables

---

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>TABLE 1-1 OS INFORMATION - ARM.....</b>           | <b>16</b> |
| <b>TABLE 1-2 COMMUNICATION METHOD -ARM .....</b>     | <b>16</b> |
| <b>TABLE 1-3 OS INFORMATION – RISC-V .....</b>       | <b>18</b> |
| <b>TABLE 1-4 COMMUNICATION METHOD - RISC-V .....</b> | <b>18</b> |
| <b>TABLE 5-1 KERNEL ELF SEGMENTS .....</b>           | <b>66</b> |
| <b>TABLE 5-2 TA ELF SEGMENTS.....</b>                | <b>66</b> |



# Chapter 1

## Introduction

## 1.1 What is MbedTEE

---

MbedTEE is an implementation of Trusted Execution Environment (TEE) for embedded devices, it's a real time operating system which comply with POSIX and GlobalPlatform, it manages the security resources and provides security services to system.

MbedTEE leverages the security architecture in the SoC, it's explicitly isolated with Rich Execution Environment (REE) and the isolation is enforced by the SoC's security technology at hardware level, remote procedure call (RPC) is introduced for the REE-TEE communication. TEE has higher security level than REE, and TEE can enable or disable the REE access to some of the hardware resources.

MbedTEE currently supports AArch32, AArch64, Riscv32, Riscv64 and MIPS32R2 architected processors (CPU).

Configuration scope note: this document focuses on architecture and security behavior. Build-time feature selection (such as Kconfig options, defconfig choices, and Buildroot integration details) is intentionally not expanded here, except where a configuration item is required to explain an architecture-visible behavior.

## 1.2 What is TA

---

A Trusted Application (TA) is a program that runs in the TEE user space and exposes security services to its clients.

A Trusted Application is command-oriented. Clients access a Trusted Application by opening a session with the Trusted Application and invoking commands within the session. When a Trusted Application receives a command, it parses the messages associated with the command, performs any required processing, and then sends a response back to the client.

A client typically runs in the Rich Execution Environment and communicates with a Trusted Application using the TEE Client API. It is then called a "Client Application". It is also possible for a Trusted Application to act as a client of another Trusted Application, using the GlobalPlatform Internal Core API. The term "Client" covers both cases.

## 1.3 Hardware Architecture

---

Before looking into MbedTEE, this section introduces some of the possible hardware architectures that the TEE may run on.

- ✧ The ARM TrustZone based architecture.
- ✧ The RISC-V based architecture.
- ✧ The MIPS based architecture.



## 1.3.1 ARM TrustZone based architecture

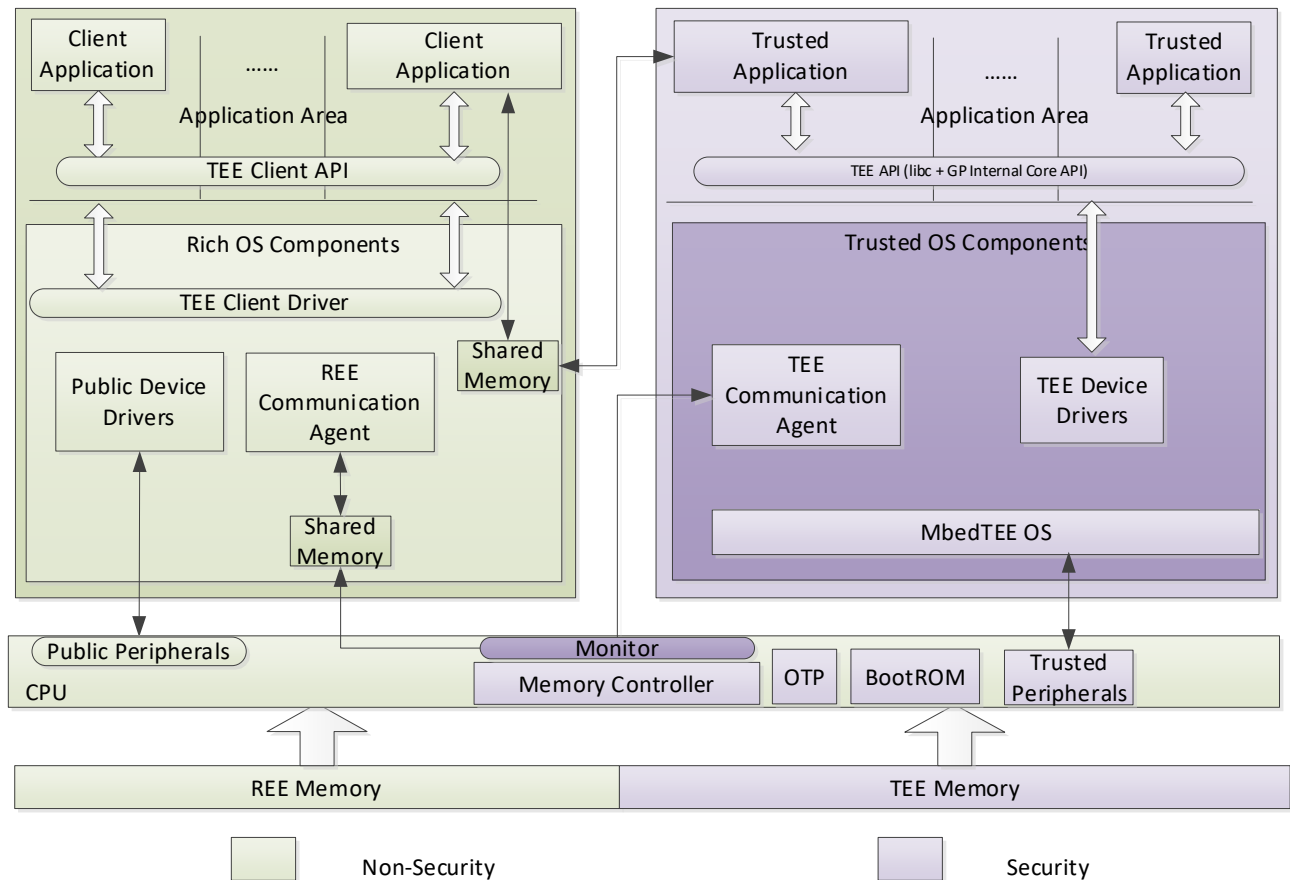


Figure 1-1 ARM TrustZone Based Architecture

The ARM processor (armv7a, armv8a and armv9a CPU) can work in non-trusted mode and trusted mode, depending on whether TrustZone is enabled. The non-trusted mode is design for the rich OS (e.g. Linux) with all non-secure applications. Whereas, the trusted mode is designed for the TEE to execute secure libraries and trusted applications.

Under the ARM TrustZone architecture, the TEE runs at the same processor (CPU) as the REE with the time-sharing scheduling mechanism. TEE and REE are explicitly isolated and this isolation is enforced by the TrustZone and SoC's security technology at hardware level, only a tiny bridge based on the SMC instruction is used for the REE-TEE communication.

The following figure takes the ARM FVP\_VE\_Cortex\_A15x4 as an example, this SoC is designed to have 4 processors in one CPU cluster, MbedTEE can run on all of these processors with SMP supported, which means SMC instruction can be executed on every processor to request the security services from MbedTEE, MbedTEE will process the requests symmetrically.

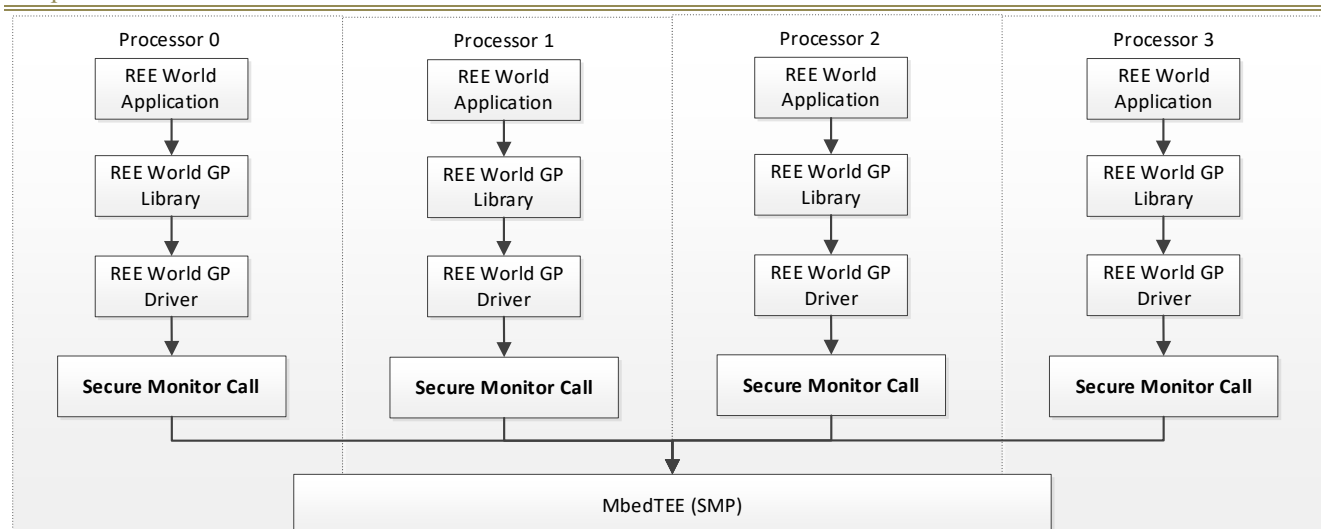


Figure 1-2 FVP\_VE\_Cortex\_A15x4 Processors

MbedTEE runs on a wide range of ARM platforms, including: VExpress (CA5/CA7/CA9/CA15/CA17/CA53/CA65/CA73+CA53/CA78/CA510+CA710), NXP i.MX7D (Cortex-A7), Raspberry Pi 4B (Cortex-A72), Aspeed AST2600 (Cortex-A7) / AST2700 (Cortex-A35), Xilinx ZCU102 (Cortex-A53) and Xilinx Versal ACAP (Cortex-A72).

Based on the above hardware architectures, ARM SoCs support the following software architecture with TEE. Table 1-1 illustrates the OS information.

Table 1-1 OS information - ARM

| Rich Execution Environment | Trusted Execution Environment |
|----------------------------|-------------------------------|
| ARM Non-Trusted (Linux)    | ARM Trusted (TEE OS)          |

Table 1-2 illustrates the communication method between different execution environments.

Table 1-2 Communication Method -ARM

|          | REE to TEE    | TEE to REE                       |
|----------|---------------|----------------------------------|
| Data     | Shared memory | Shared memory                    |
| Commands | SMC           | RPC notify via dedicated GIC SPI |

The communications between REE and TEE relies on Secure Monitor Call (SMC) instruction or IPI (inter processor interrupt), through which the CPU enters monitor mode from non-secure or secure and then fully switch the context of different worlds.

## 1.3.2 RISC-V based architecture

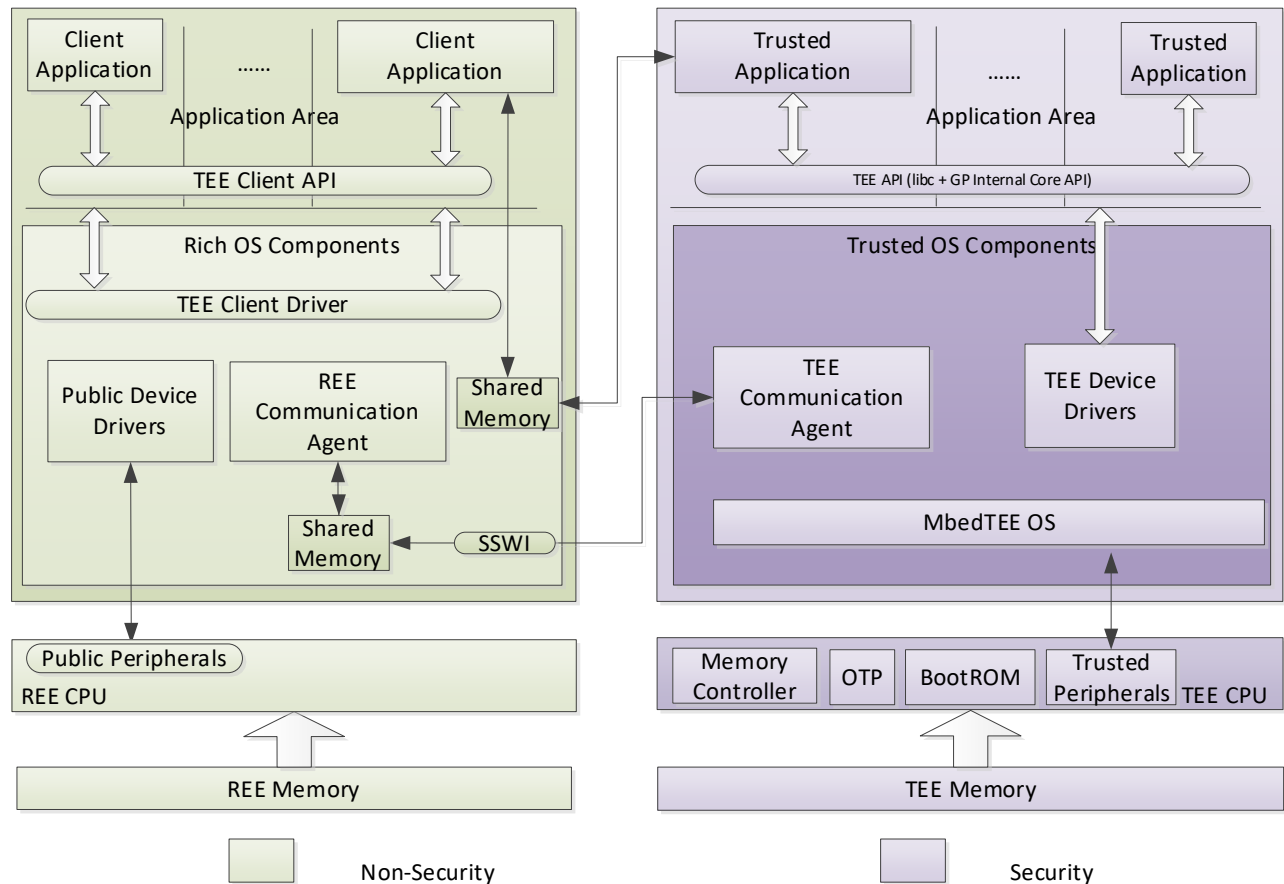


Figure 1-3 RISC-V Based Architecture

MbedTEE on RISC-V does not use an ARM TrustZone-style same-core Secure/Non-secure world switch model. In the current implementation, MbedTEE runs on dedicated TEE processors (CPU cores), while REE processors run the rich OS (e.g. Linux) with non-secure applications.

RISC-V platforms may provide different SoC-level security building blocks (for example PMP, bus firewalls, or vendor-specific attribution mechanisms such as WorldGuard), but this architecture description does not assume a single mandatory RISC-V equivalent of TrustZone.

Under this architecture, TEE and REE are explicitly isolated and the isolation is enforced by the SoC's security technology at hardware level, only a tiny bridge based on the SSWI/MSI is used for the REE-TEE communication.

The following figure takes the RISC-V32/RISC-V64 QEMU virtual platform as an example, this platform is configured to have 8 processors (e.g. `qemu-system-riscv64 -M virt -smp 8`), MbedTEE can run on all of these processors with SMP supported, which means SSWI or MSI interrupt can be served/issued on every processor to request/response the security services from MbedTEE, MbedTEE will process the requests symmetrically.

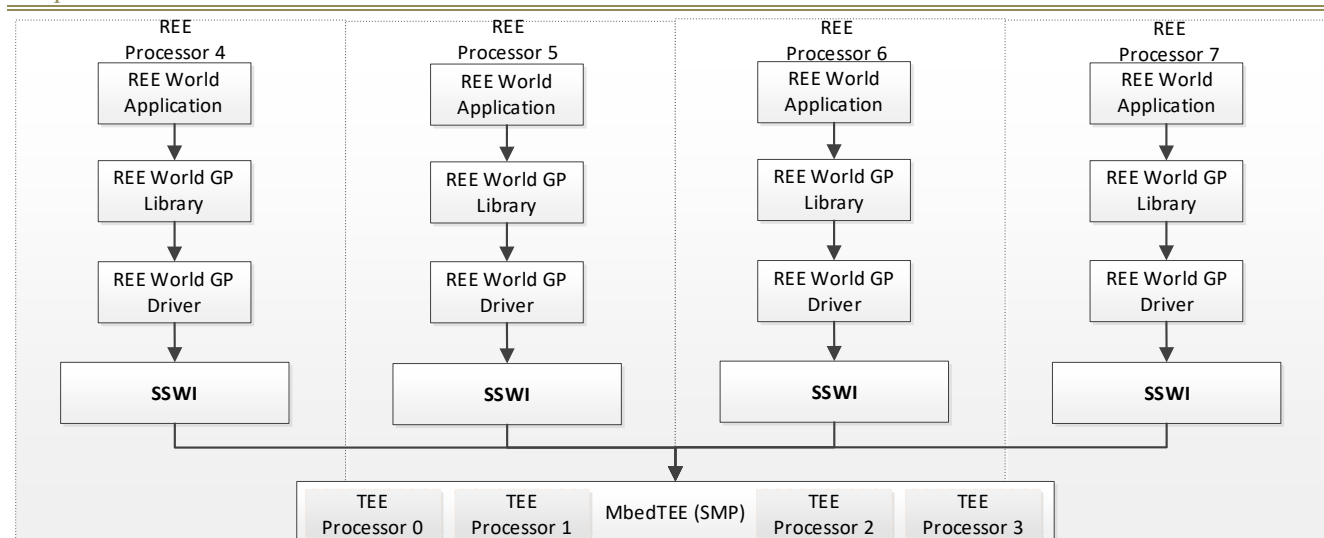


Figure 1-4 QEMU RISC-V Processors

MbedTEE runs on a wide range of RISC-V platforms, including: QEMU virt (RV32/RV64, with or without AIA/IMSIC), Andes AE350 (RV32/RV64, with or without APLIC-IMSIC), SiFive U (RV32/RV64), T-Head XiangShan KunMingHu (RV64), and Microblaze-V (RV32/RV64).

Based on the above hardware architectures, RISC-V platforms support the following software architecture with TEE. Table 1-3 illustrates the OS information.

Table 1-3 OS information – RISC-V

| Rich Execution Environment   | Trusted Execution Environment |
|------------------------------|-------------------------------|
| REE Processors (4~7) (Linux) | TEE Processors (0~3) (TEE OS) |
| REE Processors (2~3) (Linux) | TEE Processors (0~1) (TEE OS) |

Table 1-4 illustrates the communication method between different execution environments.

Table 1-4 Communication Method - RISC-V

|          | REE to TEE    | TEE to REE    |
|----------|---------------|---------------|
| Data     | Shared memory | Shared memory |
| Commands | SSWI / MSI    | SSWI / MSI    |

The communications between REE and TEE relies on software interrupt: SSWI (on platforms with PLIC/ACLINT) or MSI (on platforms with APLIC/IMSIC), through which CPUs on one side notify CPUs on the other side.

### 1.3.3 MIPS based architecture

MbedTEE currently only supports a single MIPS32R2 processor (CPU), with no SMP support and no REE communication support.

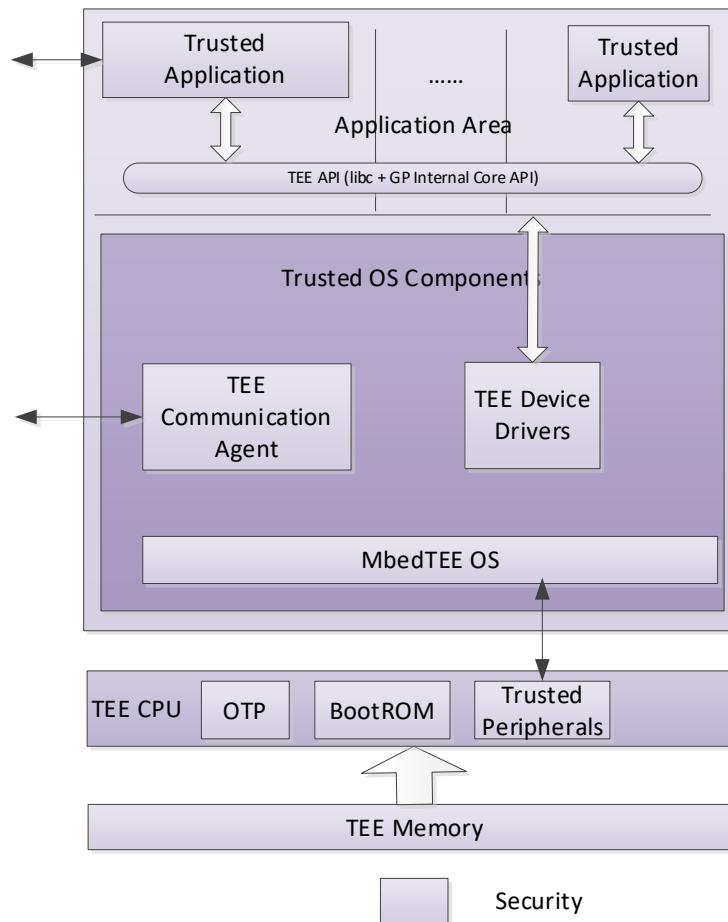


Figure 1-5 MIPS32 Based Architecture

# Chapter 2

## Overview

## 2.1 Working Model

- Client is from the REE

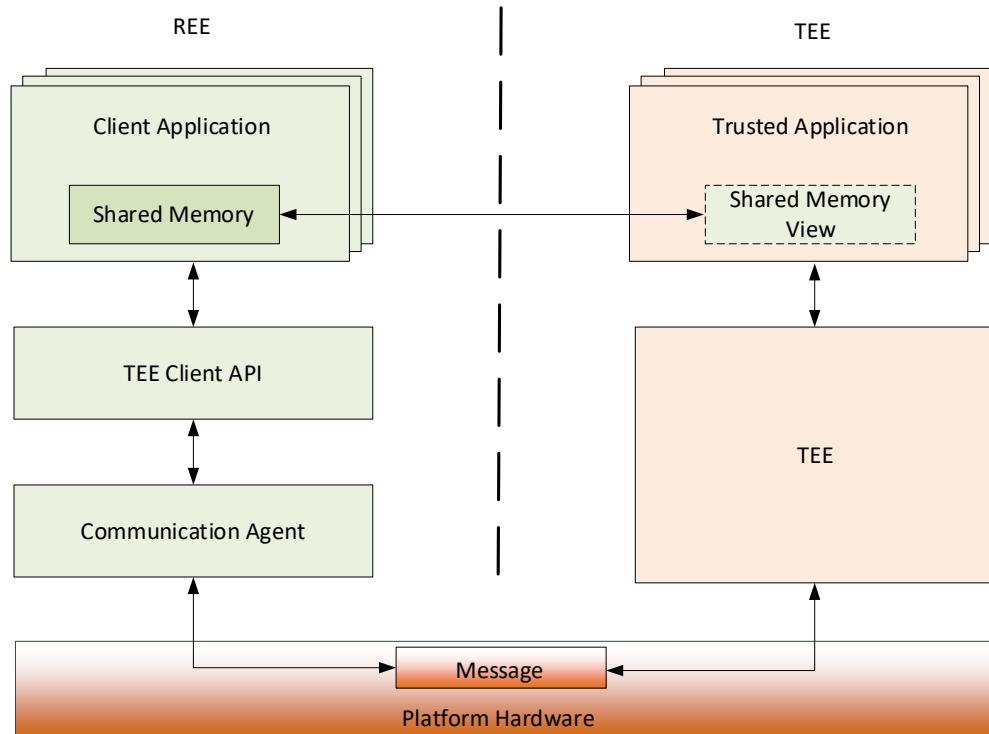


Figure 2-1 Client is from REE

- Client is another TA (instance)

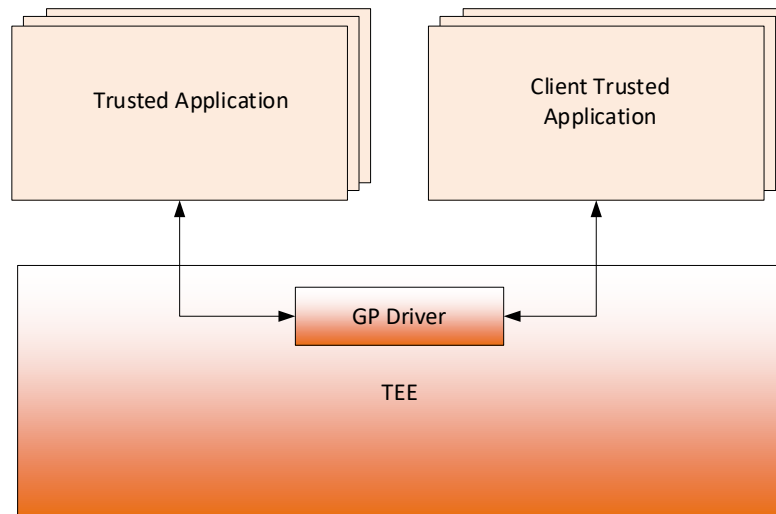


Figure 2-2 Client is another TA

## 2.2 Processor Initialization

Take the ARM TrustZone based SoC as example, RISC/MIPS flow are similar except the secondary processors (CPU) wake up method.

- Processor 0 initialization flow in TEE world:

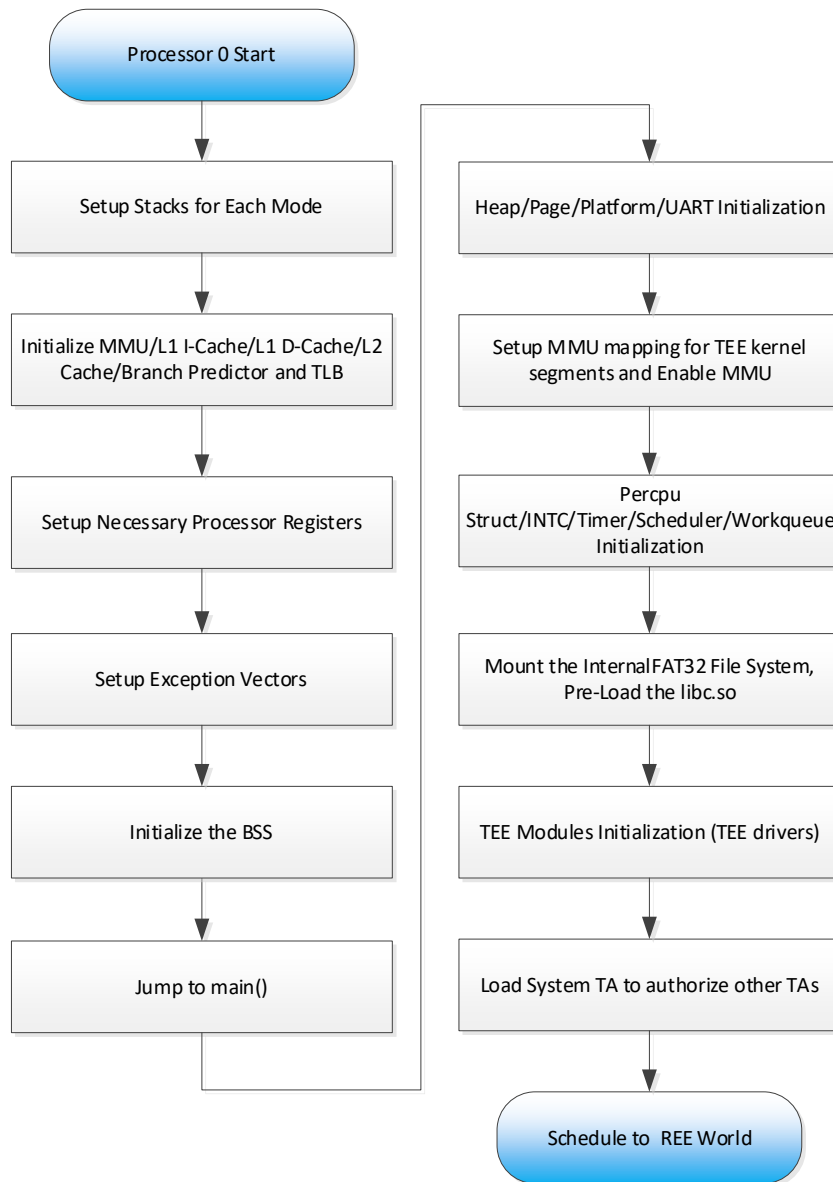


Figure 2-3 Processor 0 Initialization Flow



- Secondary processors initialization flow in TEE world:

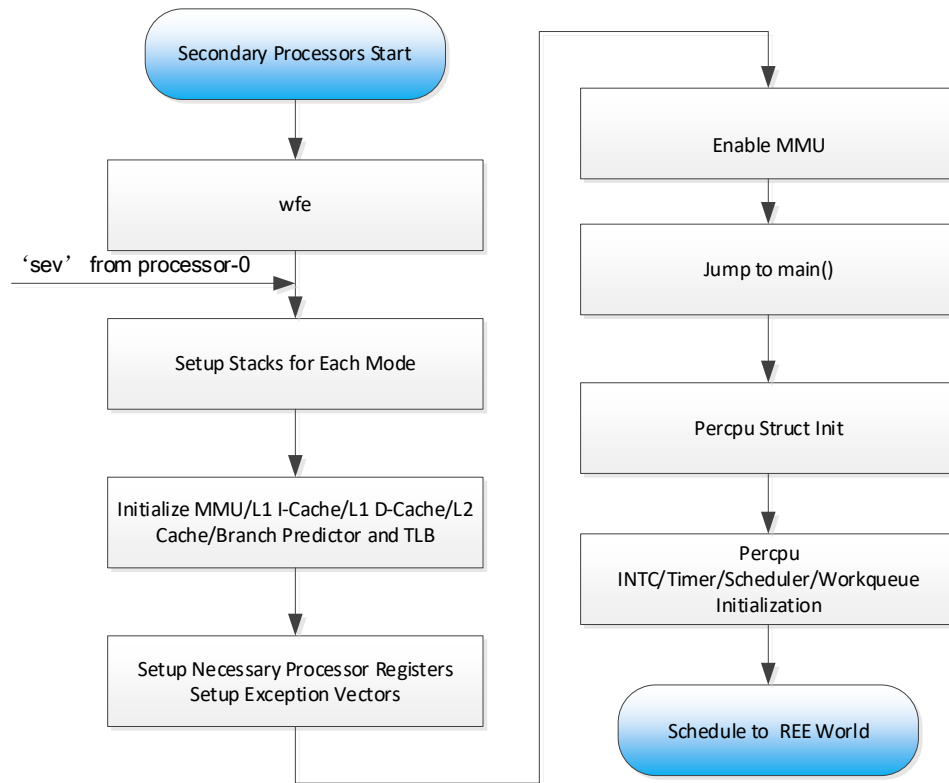


Figure 2-4 Secondary Processor Initialization Flow

## 2.3 Exception Levels

### 2.3.1 Exception Levels of ARM

The following figure illustrates the exception levels and security states in the AArch32@ARMv7-A processor. (It's a little bit different with the AArch64@ARMv8-A/ARMv9-A on the S-EL3, AArch64 has the S-EL1 for the TEE kernel and S-EL3 for the secure monitor, which the AArch32@ARMv7-A does not have the S-EL3, the TEE kernel occupies the S-EL1 alongside the secure monitor in the ARMv7-A.)

**EL0:** lowest exception level to execute the REE applications.

**EL1:** privileged exception level to execute the REE kernel.

**EL2:** privileged exception level to execute the Hypervisor.

**S-EL0:** secure exception level to execute the trusted applications.

**S-EL1:** secure exception level to execute the TEE kernel.

- **MON:** Monitor mode (AArch32 only); entered on SMC instruction or FIQ from non-secure world; executes the secure monitor routine.

**S-EL3:** highest secure exception level to execute the secure monitor routine.

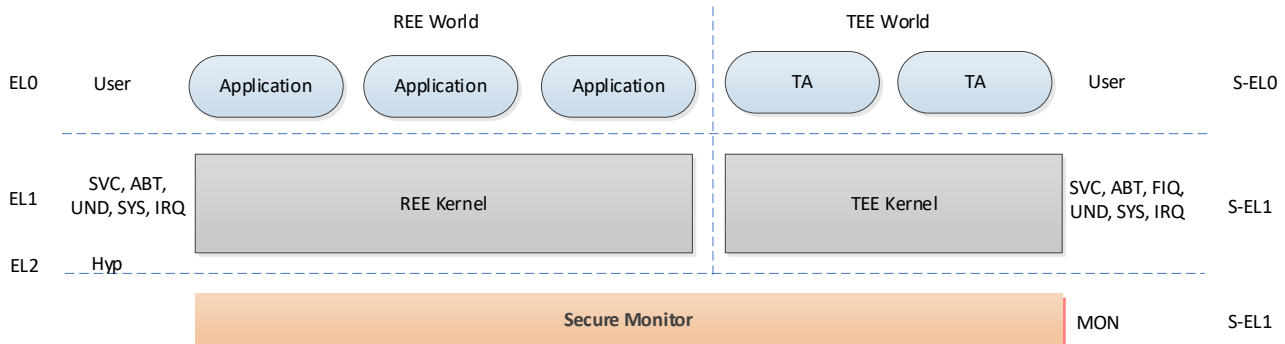


Figure 2-5 Exception Levels of AArch32@ARMv7-A

#### Processor modes in the AArch32@ARMv7-A:

**User:** non-privileged mode for most of the programs and applications.

**SVC:** Entered on reset, or when a SVC instruction is executed.

**System:** privileged mode for the operating system, sharing the registers with User mode.

**FIQ:** Entered on an FIQ interrupt exception

**IRQ:** Entered on an IRQ interrupt exception

**ABT:** Entered on a memory access exception

**UND:** Entered when an undefined instruction is executed

**MON:** Entered on an SMC instruction or on FIQ in a security-enabled processor. Monitor mode runs in the secure state only and serves as the bridge between the secure and non-secure worlds.

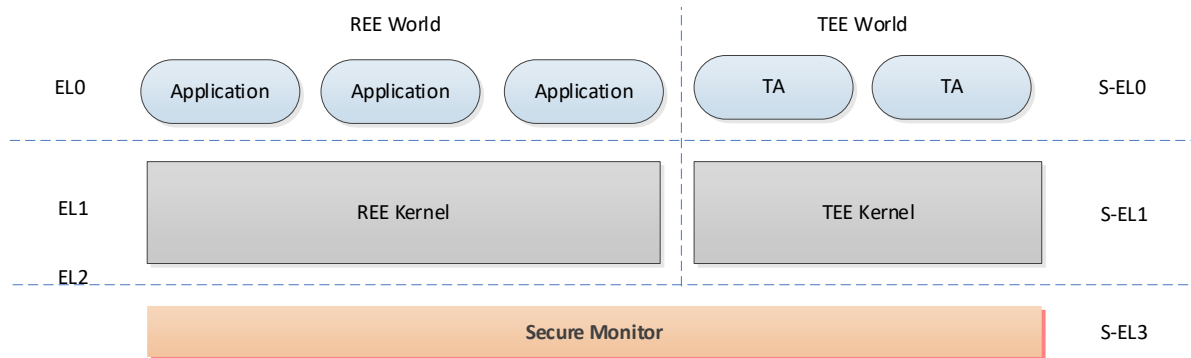


Figure 2-6 Exception Levels of AArch64@ARMv8-A/ARMv9-A

## 2.3.2 Exception Levels of RISC-V

**Level 0:** User exception level to execute the applications.

**Level 1: Supervisor** exception level to execute the kernel.

**Level 2:** Reserved.

**Level 3: Machine** exception level, highest privileges.

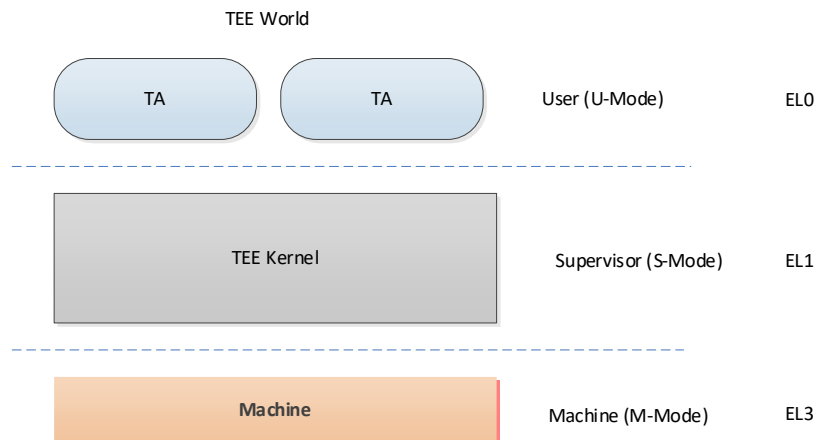


Figure 2-7 Exception Levels of RISC-V

## 2.3.3 Exception Levels of MIPS

**User Mode:** User exception level to execute the applications.

**Kernel Mode:** **Kernel** exception level to execute the kernel. The core enters kernel mode both at reset and when an exception is recognized.

**Supervisor Mode:** Not in use.

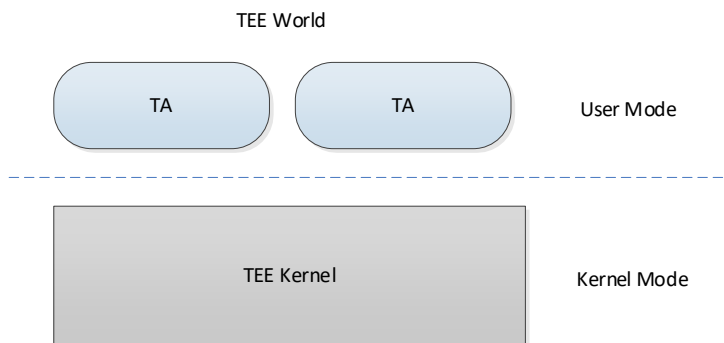


Figure 2-8 Exception Levels of MIPS

## 2.4 Multi-threading

---

Most modern operating systems provide multi-threading support, and MbedTEE is no different. MbedTEE supports multi-threads as they can provide concurrency or parallelism on the single or multiple processor (CPU) systems.

MbedTEE modules (TA or kernel modules) can attain parallelism by spawning some operations in the background, TA or kernel modules achieve this by delegating these operations to the corresponding user or kernel threads, these threads are independent sequences of execution corresponding to their mission.

**There are diverse definitions of user and kernel threads across different operating systems, the following sections introduce what they are under the MbedTEE operating system.**

### 2.4.1 Thread

---

A thread is an entity within a process that can be scheduled for execution. Each thread has its own scheduling attribute (e.g. scheduling policy and priority), thread local storage, a unique thread identifier, and a set of structures the operating system will use to save/restore the thread context in scheduling. The thread context includes the thread's set of machine registers, the kernel stack and the user stack in its process's address space.

### 2.4.2 Process

---

A process is one that only runs at the user space and runs only the TA's code. Each process is started with a single thread, often called the primary thread, but can create additional threads from any of its threads, each process must have at least one thread of execution. Each process provides the resources needed to execute a program, including a virtual address space, executable code, heap, opened handles to system objects (e.g. file descriptors), device ACL, IPC ACL, a unique process identifier, process configuration (e.g. name and UUID). The process's resources are visible to all of its threads, meaning that threads within the same process can share resources with one another. The process resource is totally not visible to other processes, the resource sharing between processes must go through the explicit IPC.

### 2.4.3 User thread

---

A user thread is one that runs at the user space, but it can call into kernel space by issuing system call, it's still considered a user thread even though it can execute kernel code.

MbedTEE provides complete POSIX thread support in the user space; each process (TA) can create its own thread(s), and newly created threads are only visible within the owner process (TA).

### 2.4.4 Kernel thread

---

A kernel thread is one that only runs at the kernel space and runs only the kernel code, it's created by the operating system or kernel modules and it's not associated with any user thread.

Both the user and kernel threads are scheduled by the kernel scheduler.

## **2.4.5 Thread ID**

---

Thread unique identifier within the whole MbedTEE.

## **2.4.6 POSIX Thread ID**

---

Thread unique identifier within its owner process (TA).

## **2.4.7 Process ID**

---

Process unique identifier within MbedTEE, equal to its primary thread ID.

## 2.5 Scheduler

---

In order to support multi-threading, MbedTEE supports the POSIX standard scheduler to control the execution of each thread. Each thread has its own associated scheduling policy and priority. Associated with each policy is a priority range. Each policy specifies the minimum and maximum priorities for that policy. The priority ranges for each policy may overlap the priority ranges of other policies.

To balance the load across processors (CPU) running in the same MbedTEE instance, MbedTEE scheduler supports the Symmetric Multi-Processing (SMP) mechanism.

MbedTEE provides the support of following scheduler policies:

### 2.5.1 SCHED\_FIFO

---

Generally, `SCHED_FIFO` is a real-time scheduling policy known as "first come, first served." This strategy allows real-time processes to run until a higher priority thread arrives or they voluntarily yield the processor (CPU). `SCHED_FIFO` is suitable for threads that require a strict response time because it ensures that once a thread is executed, it will not be preempted by other threads of same priority.

MbedTEE supports the `SCHED_FIFO` policy, threads scheduled under this policy are chosen from a thread ready queue that is ordered by the time its threads have been on the queue without being executed; generally, the head of the queue is the thread that has been on the queue the longest time, and the tail is the thread that has been on the queue the shortest time.

- ✧ When a `SCHED_FIFO` thread starts running, it continues to run until it voluntarily gives up CPU or is preempted by a higher priority thread.
- ✧ If multiple `SCHED_FIFO` threads of the same priority are ready to run, the `SCHED_FIFO` policy requires that the currently running thread actively yield the CPU before subsequent threads of the same priority can run.
- ✧ The thread becomes the tail of the thread ready queue under the following cases:
  - ✧ When a running thread yield CPU,
  - ✧ When a blocked thread becomes a runnable thread,
  - ✧ When the priority is changed.

When using `SCHED_FIFO`, be aware that such threads can occupy the CPU for a long time, affecting the execution of other threads on the system. Therefore, such threads should be designed to ensure that they can yield the CPU in a reasonable amount of time to avoid excessive use of system resources.

### 2.5.2 SCHED\_RR

---

MbedTEE supports the `SCHED_RR` policy, `SCHED_RR` is a real-time scheduling policy that manages threads on a Round-Robin basis. This policy ensures fairness by allowing threads with the same priority to take turns using CPU resources in the order of time slices. When a thread runs out of time slices, it is placed at the end of the ready queue, waiting for the next dispatch.

The effect of this policy is to ensure that if there are multiple `SCHED_RR` threads at the same priority, one of them does not monopolize the CPU. An application should not rely only on the use of `SCHED_RR` to ensure application progress among

multiple threads if the application includes threads using the `SCHED_FIFO` policy at the same or higher priority levels or `SCHED_RR` threads at a higher priority level.

## 2.5.3 `SCHED_OTHER`

---

POSIX defines the `SCHED_OTHER` policy, which is implementation-defined, allowing the operating system to define its own specific scheduling policy to improve portability.

MbedTEE `SCHED_OTHER` is a dynamic-priority policy; the priority will be automatically decreased after the thread has consumed a certain CPU time slice. When the priority decreases to the minimum, MbedTEE scheduler will reset its priority to the original value, and so on. This policy ensures that no thread with the `SCHED_OTHER` policy can monopolize the CPU. Most TAs will use this policy and this is the default policy when creating a thread.

## 2.5.4 Priority

---

MbedTEE scheduler manages both the kernel and user thread execution; the kernel and user threads have different priority ranges, and most kernel threads have higher priority than user threads. The priority differences are listed as below, higher value corresponding to higher priority.

- ✧ 0 ~ 63 for the kernel thread priority
- ✧ 0 ~ 39 for the user thread priority
- ✧ Default kernel thread priority is 44
- ✧ Default user thread priority is 16

A kernel thread can set its priority even lower than that of a user thread, depending on the thread's mission.



## 2.6 Exception Handling

---

Exception is a system event or condition that requires halting normal execution and instead executing a dedicated software routine known as the exception handler. After the exception has been handled, the operating system resumes the context before taking the exception if there is no context switch required. (refer to the following chapter for the context switch).

**MbedTEE kernel catches all the exceptions and handles all of them to avoid the undefined behaviors of the TEE execution.**

Here takes the AArch32 exceptions as an example (4 kinds of exceptions):

- Interrupt:
  - ✧ FIQ: According to the MbedTEE design, the FIQ is dedicated for the TEE world, and all FIQs are routed to the monitor mode. The FIQ is not visible to the REE world, REE world can't influence any FIQ configurations and exceptions.
  - ✧ IRQ: The IRQ is dedicated for the REE world, but it may occur during the MbedTEE execution. In the case of an IRQ occurring during MbedTEE execution, MbedTEE just switches the context to REE instead of responding to this IRQ, which means the REE IRQ can interrupt MbedTEE execution and MbedTEE just forwards the IRQ to REE. A substituted solution is that MbedTEE masks the IRQ during MbedTEE execution; as a result, the IRQ will never be able to interrupt MbedTEE, but the IRQ latency time will be terrible.
- Aborts:
  - ✧ Prefetch aborts on the instruction fetch
  - ✧ Data aborts on the data access
  - ✧ Undefined aborts
- **Reset:** Reset is the highest priority exception and can't be masked.
- Software Generated:
  - ✧ The supervisor call (SVC) enables the user code to request the kernel service.
  - ✧ The secure monitor call (SMC) enables the REE to request the TEE service.

# Chapter 3

## Context Switch

## 3.1 Context Switch of ARM

Context switch may happen at any time during processor (CPU) execution, and may be triggered by the software interrupt or the external interrupt.

There are two kinds of context switches under the ARM TrustZone based MbedTEE architecture, one is the REE-TEE world context switch, another one is the thread context switch inside the MbedTEE.

The following sections illustrate the different handling manners of different cases inside MbedTEE.

### 3.1.1 REE SMC

REE can request the TEE service through the SMC, there are two kinds of function calls can be issued by REE: the synchronous and the asynchronous calls.

#### 3.1.1.1 Synchronous Call

The AArch32 synchronous call is used for the short time request, with this type, the monitor routine does not wake up any MbedTEE threads to respond to this call, it handles this kind of calls directly with the FIQ and IRQ masked. Only the REE-TEE world contexts are switched under this type, no context switch of MbedTEE threads.

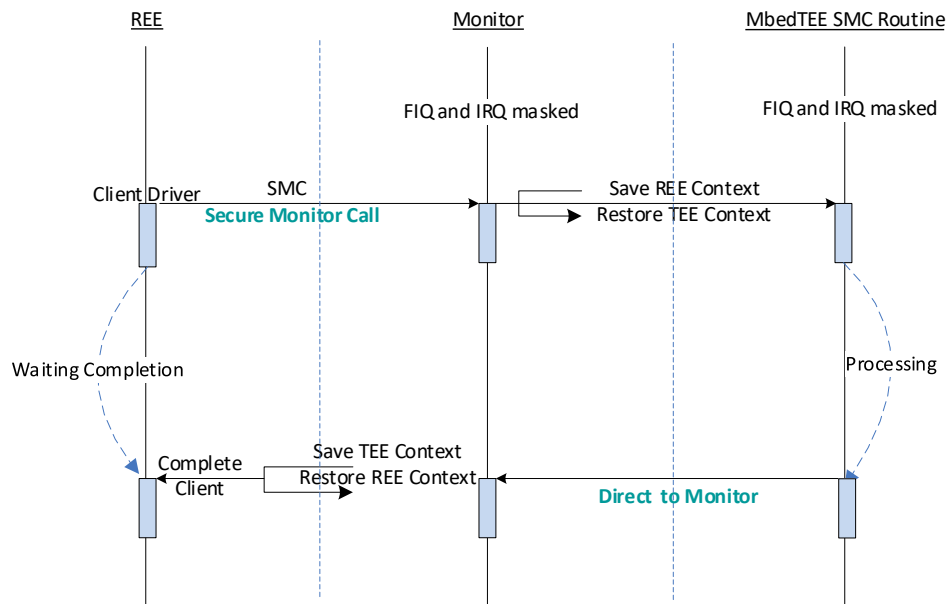


Figure 3-1 Synchronous SMC Call – AArch32

The AArch64 synchronous call is different with AArch32, AArch64 monitor delegates the EL3 SMC to EL1 SGI, then the synchronous call can be handled in EL1 MbedTEE SGI routine properly.

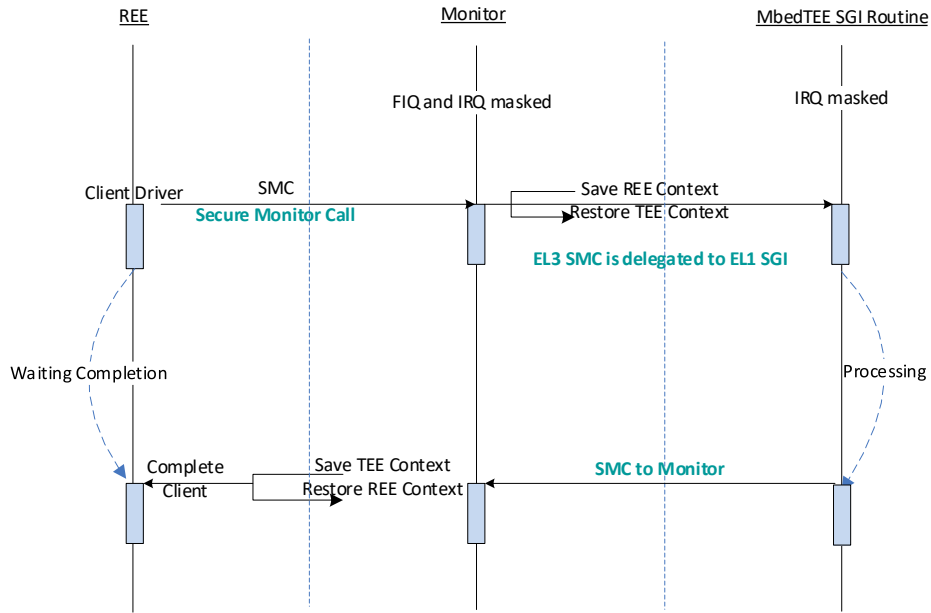


Figure 3-2 Synchronous SMC Call – AArch64

### 3.1.1.2 Asynchronous Call

The asynchronous call is used for handling long time request which may lead MbedTEE wait, with this type, the monitor routine (or EL1 SGI routine @ AArch64) wakes up a MbedTEE kernel thread (rpc-workqueue) to respond to this call by enqueueing work to the rpc-workqueue. The REE-TEE world context switch and the MbedTEE thread context switch are both involved.

For the AArch32 SMC:

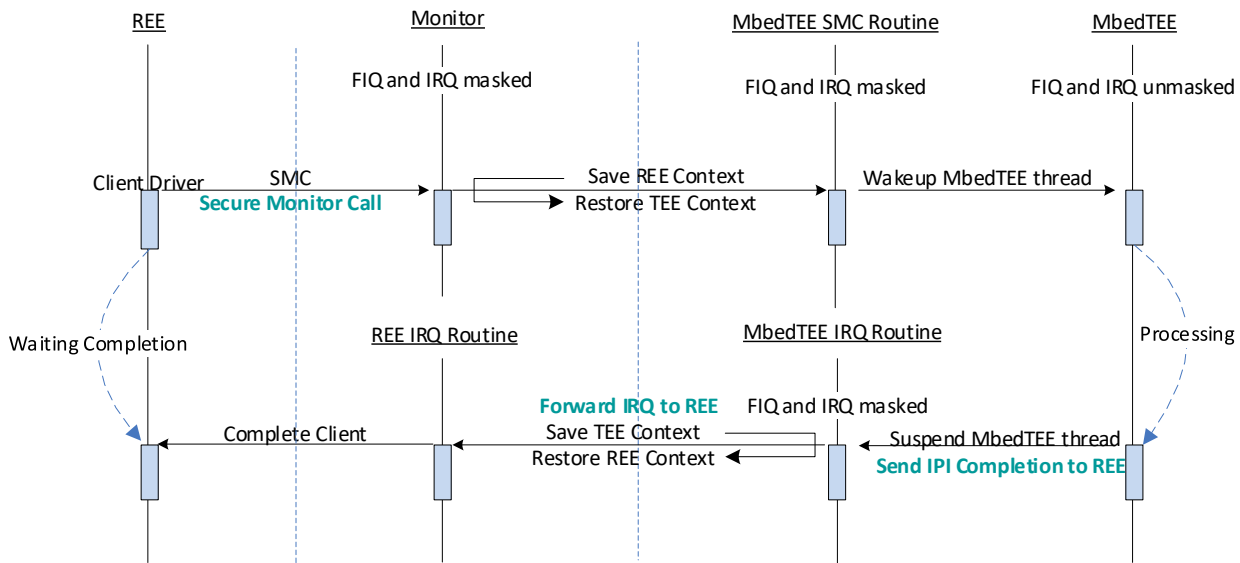


Figure 3-3 Asynchronous SMC Call – AArch32

For the AArch64 SMC:

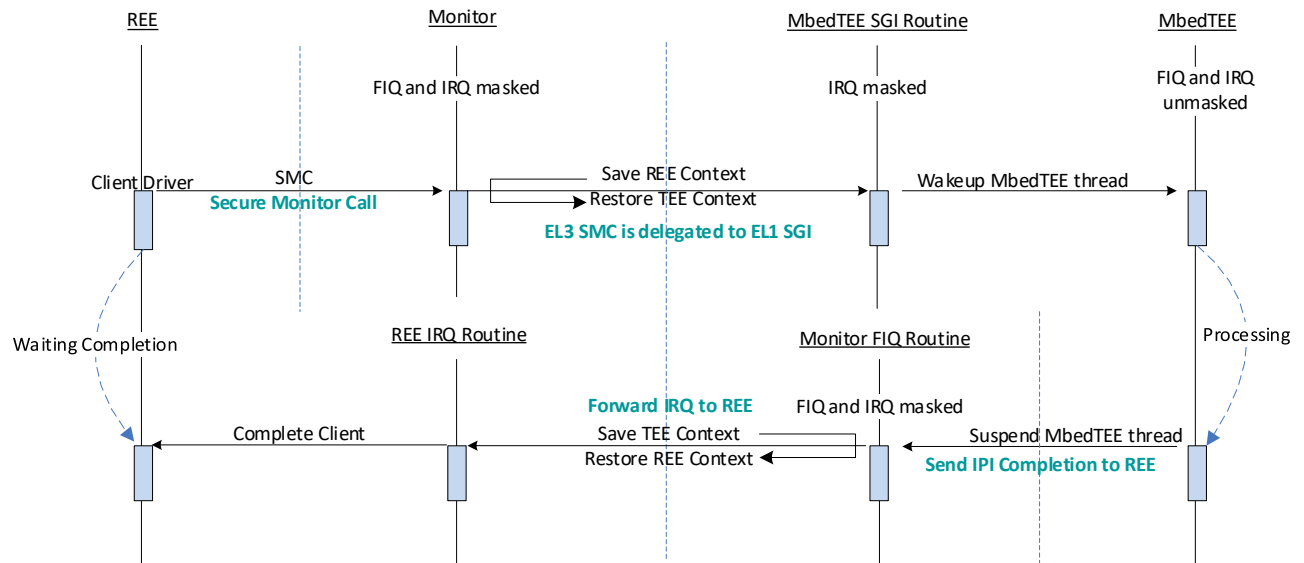


Figure 3-4 Asynchronous SMC Call – AArch64

### 3.1.2 TEE SMC

Normally the MbedTEE kernel is working at the AArch32 System mode or AArch64 EL1T, when the MbedTEE kernel needs to request the monitor services, it can also issue the SMC to enter the monitor mode. Currently this kind of TEE SMC (SMC Issued from MbedTEE internal) is only used for the secondary processors power up and power down.

### 3.1.3 AArch32 SVC

AArch32 SVC is the software generated supervisor call to request the kernel service (system call). The following figure illustrates the AArch32 context switch of both the normal SVC and scheduler SVC.

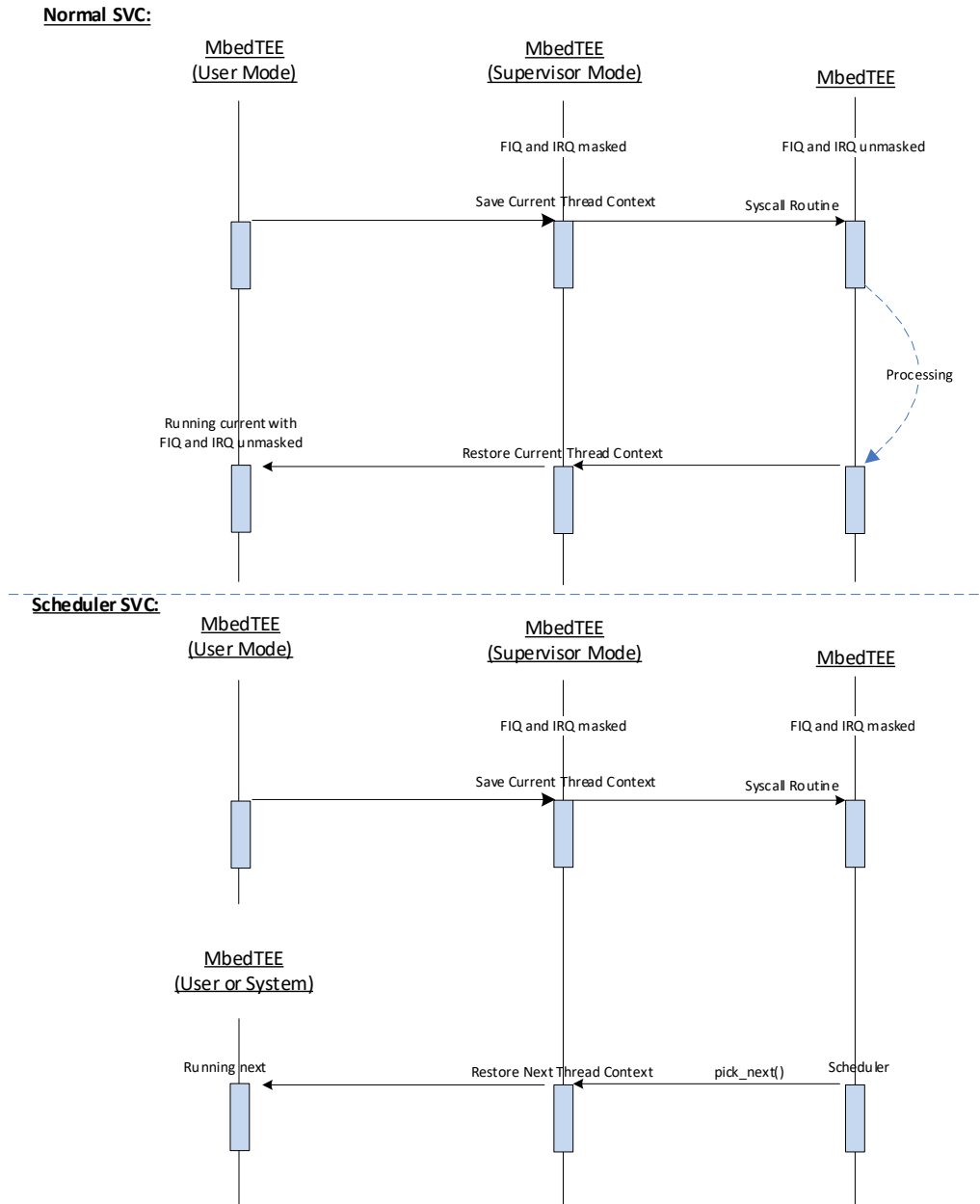


Figure 3-5 System Call – AArch32

### 3.1.4 AArch64 SVC

AArch64 SVC is the software generated supervisor call to request the kernel service (system call). The following figure illustrates the AArch64 context switch of both the normal SVC and scheduler SVC.

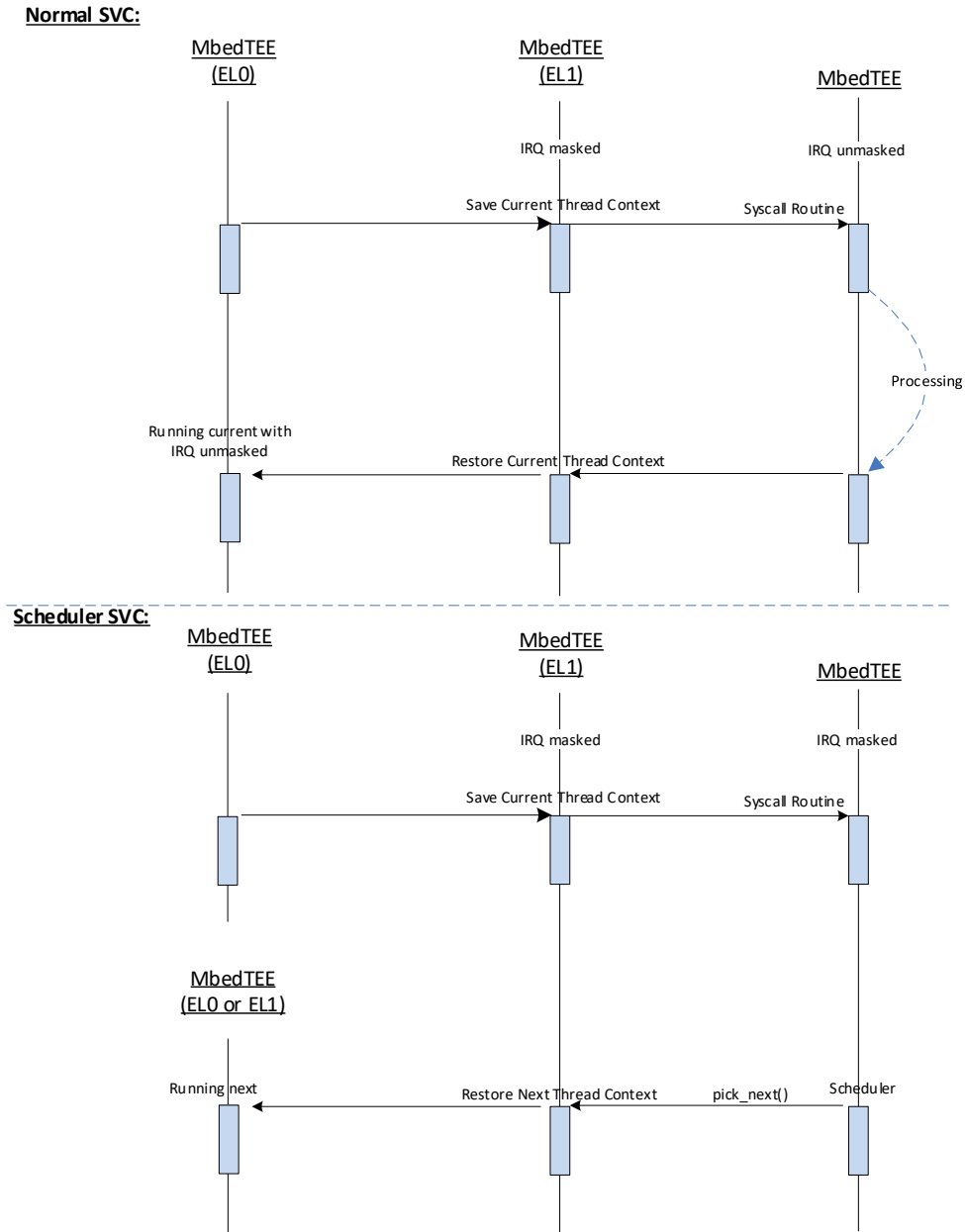


Figure 3-6 System Call – AArch64

## 3.1.5 AArch32 Interrupt

### 3.1.5.1 FIQ Raised during REE Execution

The AArch32 FIQ is dedicated for the MbedTEE and the FIQs are all routed to monitor mode, so when a FIQ is raised during REE execution, the monitor receives this FIQ and switches the context to the MbedTEE, then the MbedTEE corresponding FIQ routine can respond this FIQ.

- If it is a normal FIQ, it isn't related to any scheduler functionalities.

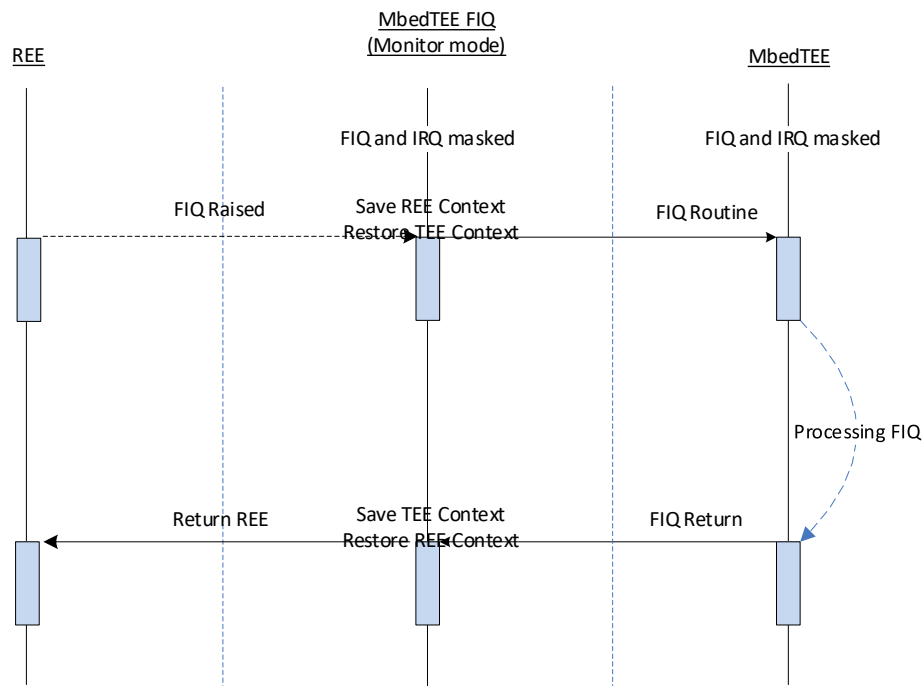


Figure 3-7 FIQ Interrupts the REE - AArch32

- If it is a FIQ related to MbedTEE scheduler. (e.g. scheduler timer FIQ)

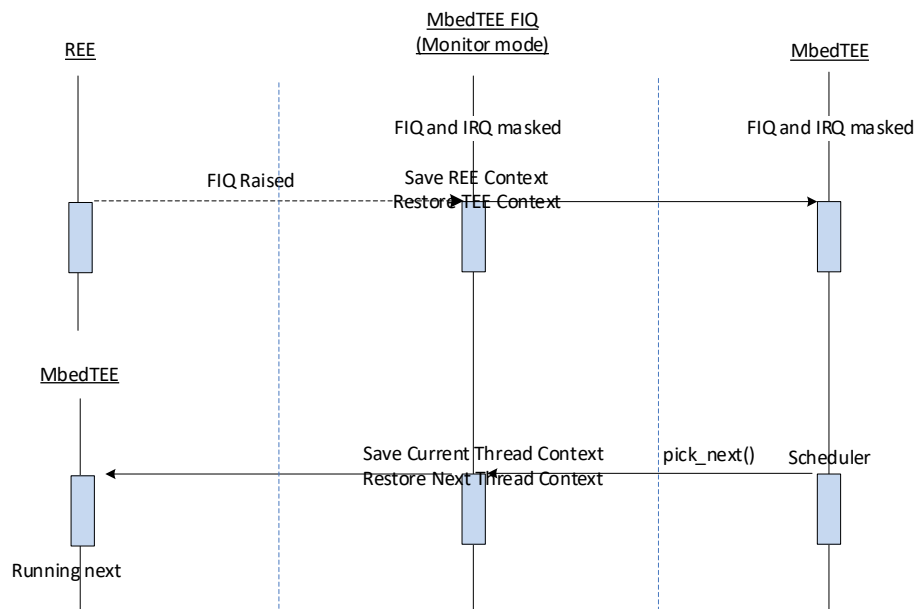


Figure 3-8 FIQ Interrupts the REE (Scheduled) - AArch32



### 3.1.5.2 FIQ Raised during TEE Execution

Due to the AArch32 FIQ is dedicated for the MbedTEE, so the handler logic is quite simple when the FIQ is raised during MbedTEE execution. Refer to the following figure to know the details of the normal FIQ and scheduler FIQ handler logics.

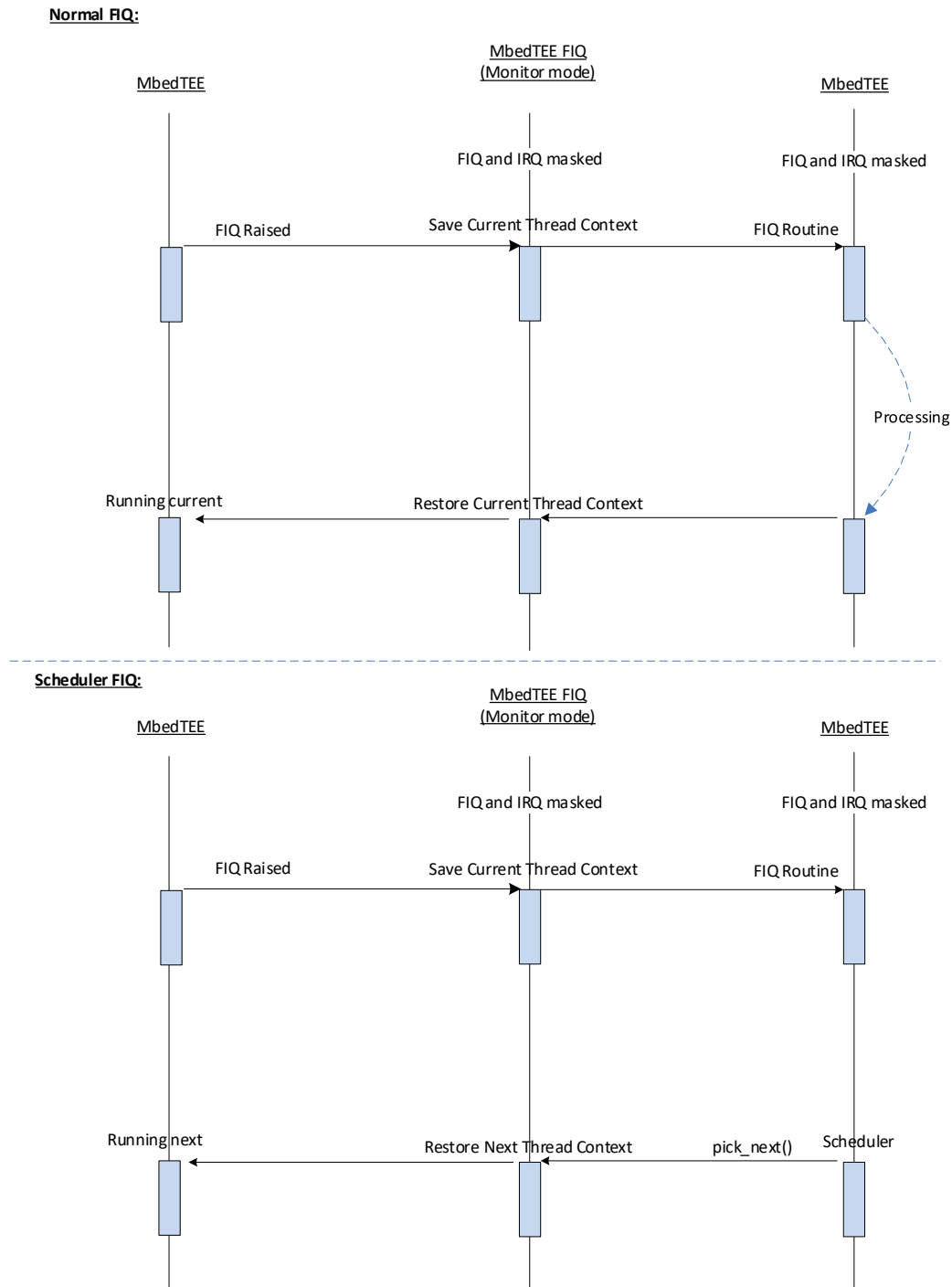


Figure 3-9 FIQ - AArch32

### 3.1.5.3 IRQ Raised during TEE Execution

---

The AArch32 IRQ is dedicated for the REE and the IRQ may occur during the TEE execution, MbedTEE just switches the context to REE instead of responding this IRQ.

After the AArch32 processor context is switched to REE, REE receives this IRQ and handles it immediately.

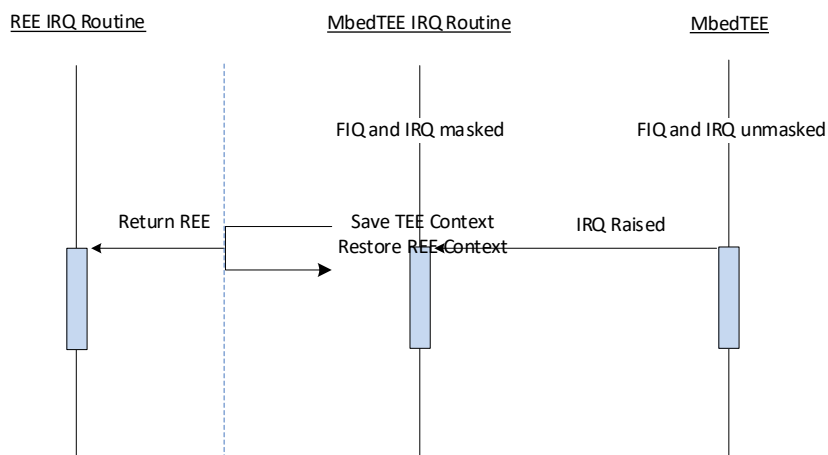


Figure 3-10 MbedTEE Forwards IRQ to REE - AArch32

### 3.1.5.4 IRQ Raised during REE Execution

---

The AArch32 IRQ is dedicated for the REE, so there is no context switch in MbedTEE when the IRQ is raised during REE execution.

## 3.1.6 AArch64 Interrupt

### 3.1.6.1 FIQ

The AArch64 FIQ is dedicated for the monitor mode, Monitor FIQ will be asserted on the following cases:

- ✧ TEE IRQ raised during REE execution
- ✧ REE IRQ raised during TEE execution

For case 1, when a TEE IRQ is raised during REE execution, the processor jumps to the monitor FIQ exception entry, then the monitor switches the context to the MbedTEE, the MbedTEE corresponding IRQ routine will respond this IRQ.

For case 2, when a REE IRQ is raised during TEE execution, the processor jumps to the monitor FIQ exception entry, then the monitor switches the context to the REE, the REE corresponding IRQ routine will respond this IRQ.

- If it is a normal TEE IRQ, it isn't related to any scheduler functionalities.

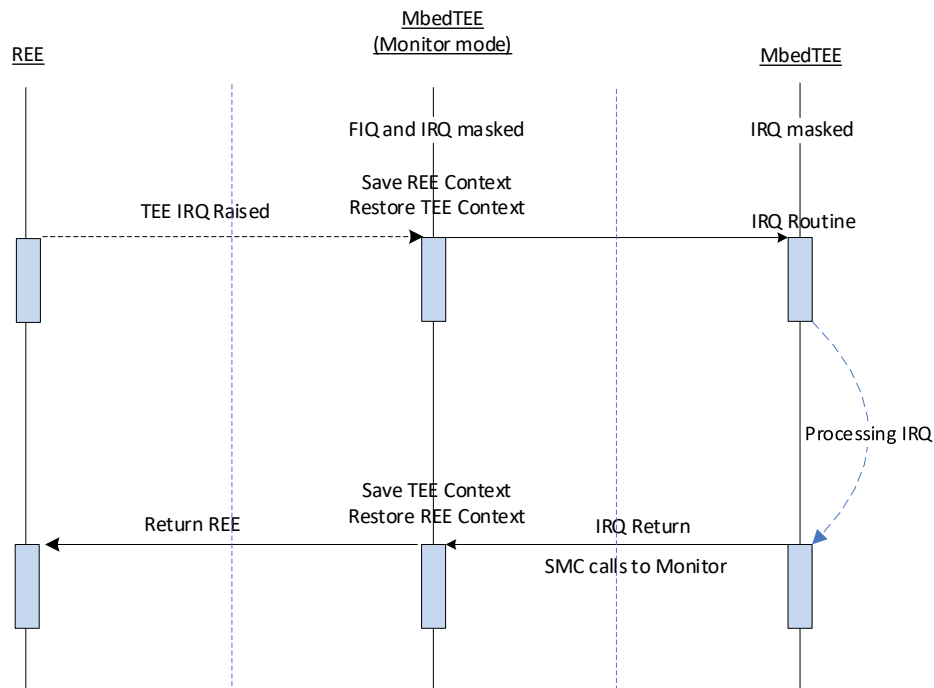


Figure 3-11 TEE IRQ Interrupts the REE - AArch64

- If it is a TEE IRQ related to MbedTEE scheduler. (e.g. scheduler timer IRQ)

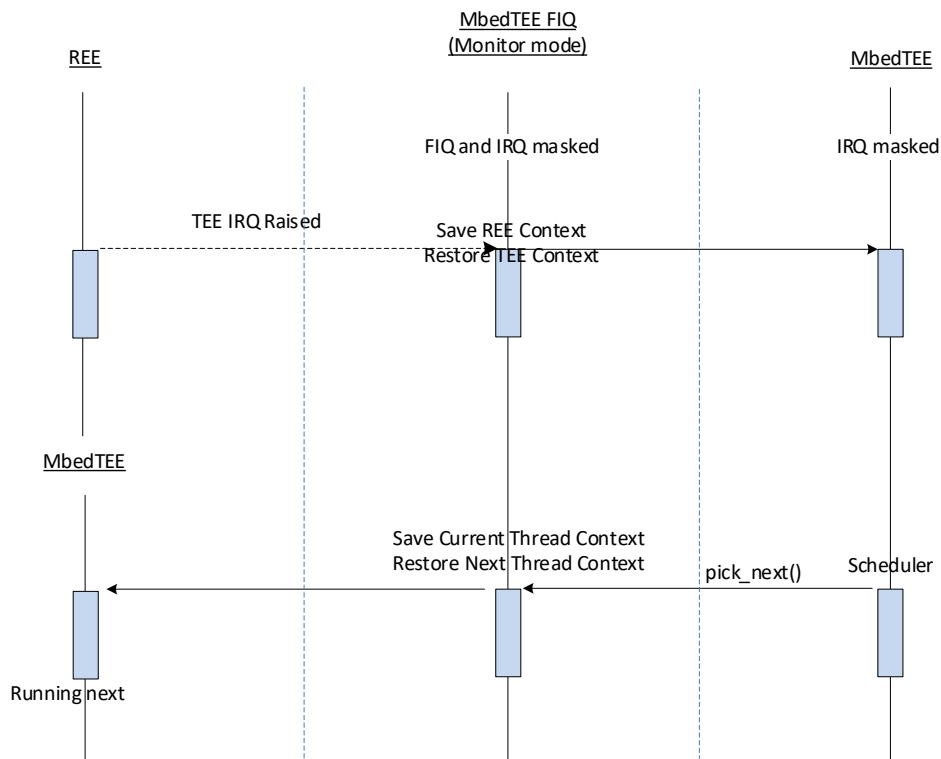


Figure 3-12 TEE IRQ Interrupts the REE (Scheduled) - AArch64

- REE IRQ raised during TEE execution.

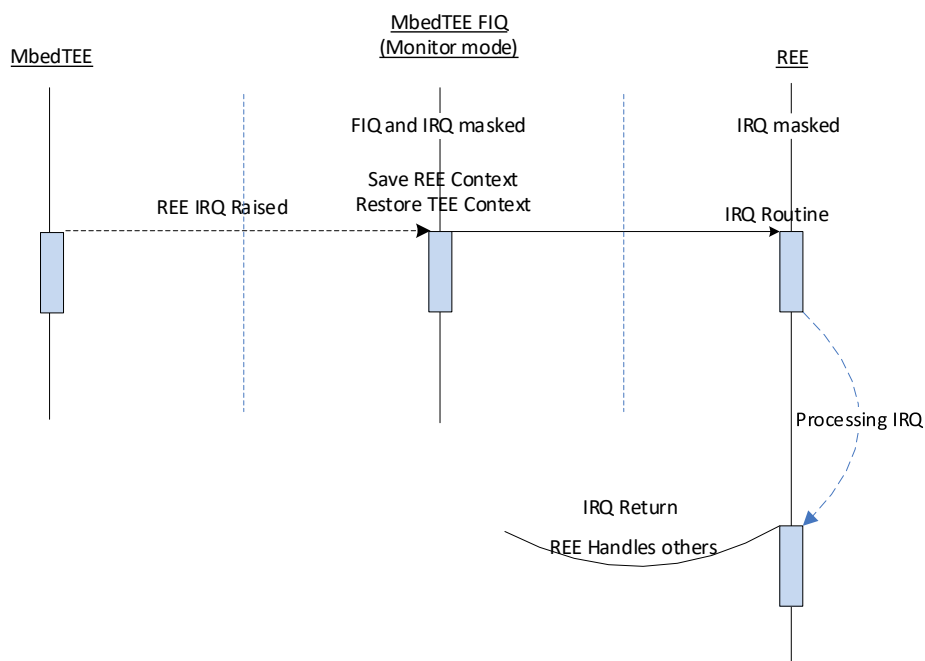
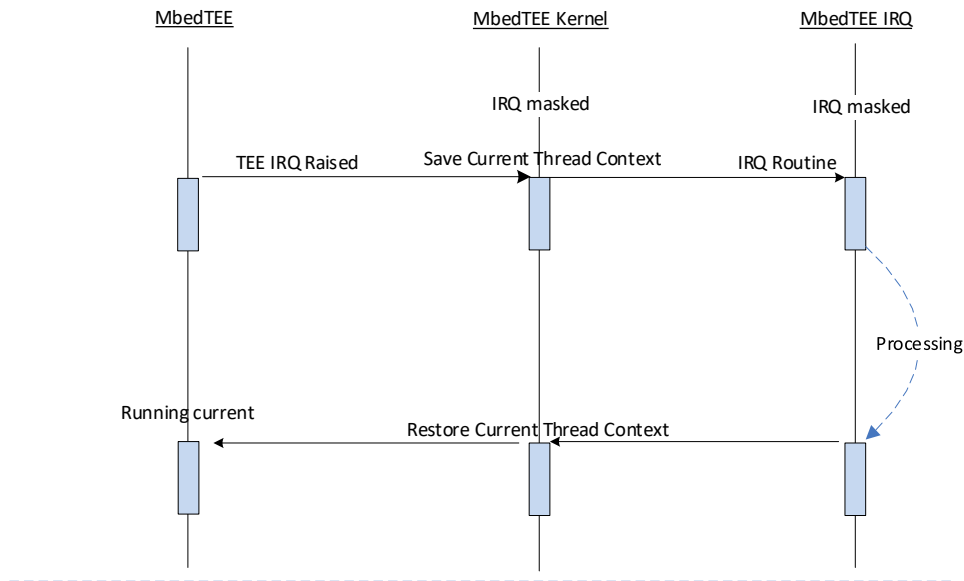


Figure 3-13 REE IRQ Interrupts the TEE - AArch64

### 3.1.6.2 TEE IRQ Raised during TEE Execution

Refer to the following figure to know the details of the normal TEE IRQ and scheduler TEE IRQ handler logics.

#### Normal IRQ:



#### Scheduler IRQ:

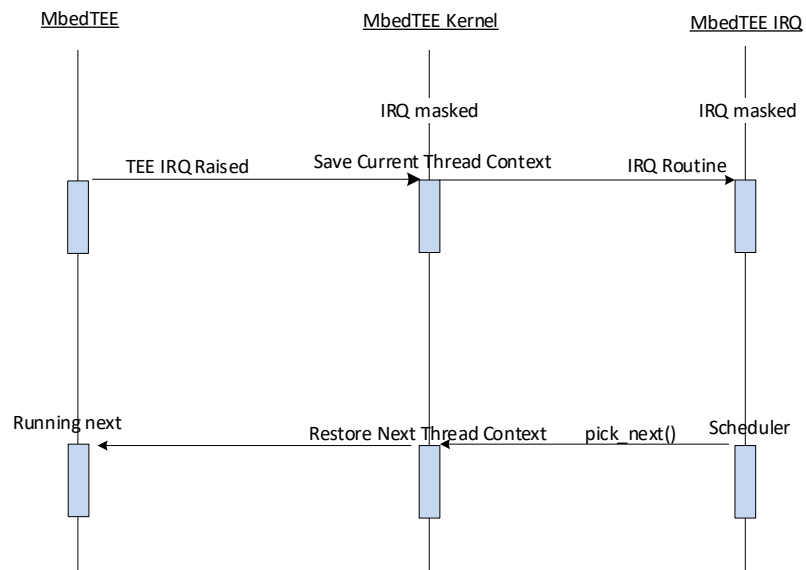


Figure 3-14 TEE IRQ - AArch64

### 3.1.6.3 REE IRQ Raised during REE Execution

There is no context switch in MbedTEE when the REE IRQ is raised during REE execution.

## 3.2 Context Switch of RISC-V

Context switch may happen at any time during RISC-V processor (CPU) execution, and may be triggered by the software interrupt or the external interrupt.

The following sections illustrate the different handling manners of different cases inside MbedTEE.

### 3.2.1 System Call

RISC-V supports the ECALL instruction to make a request to the supporting execution environment, MbedTEE trusted application uses such software generated environment call to request the kernel service, which means the system call. The following figure illustrates the context switch on both the normal system call and scheduler system call.

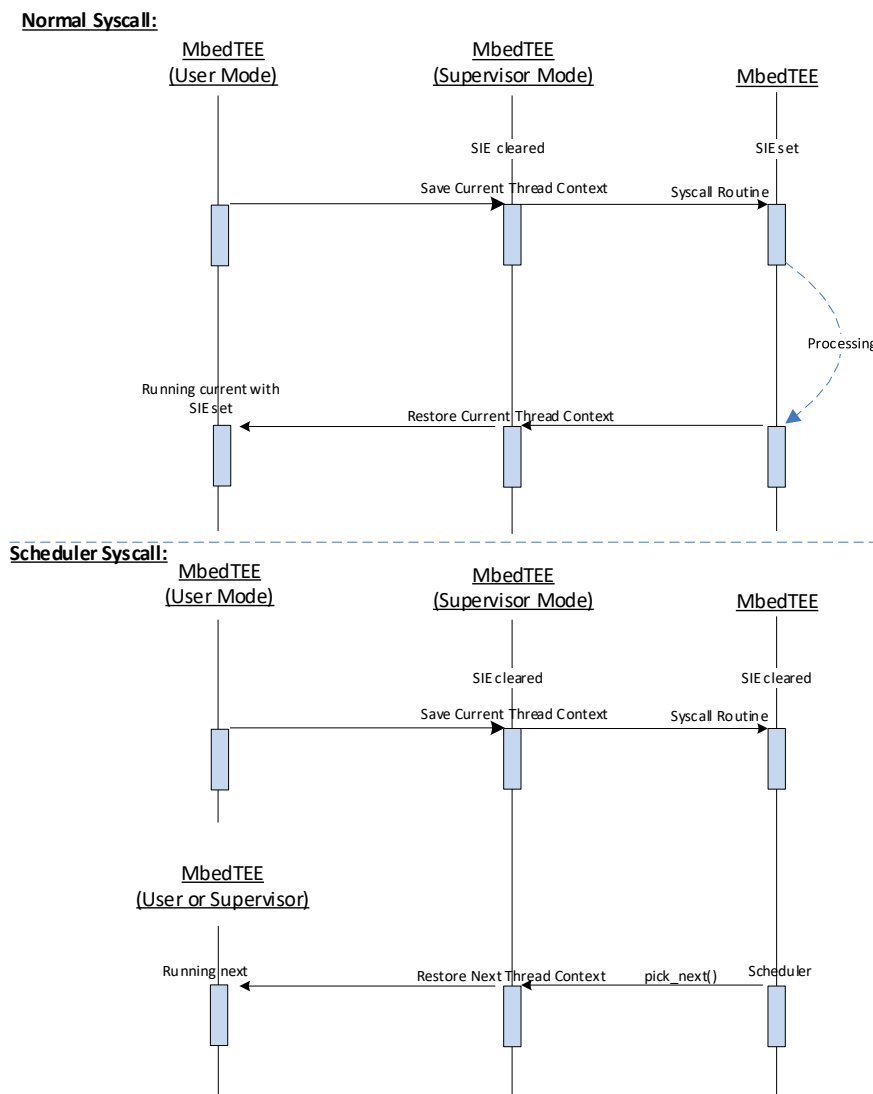
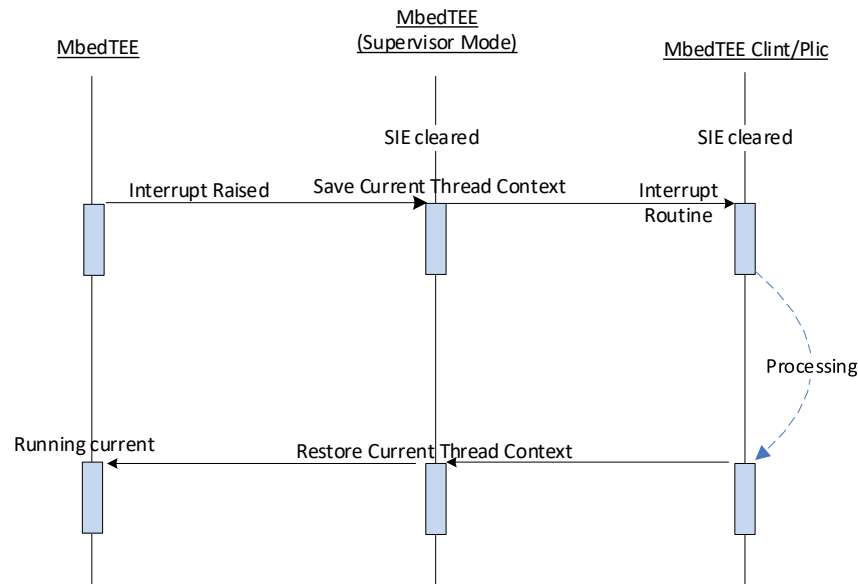


Figure 3-15 System Call - RISC-V

## 3.2.2 Interrupt

Refer to the following figure to know the details of the RISC-V normal interrupt and scheduler interrupt handler logics.

### Normal Interrupt:



### Scheduler Interrupt:

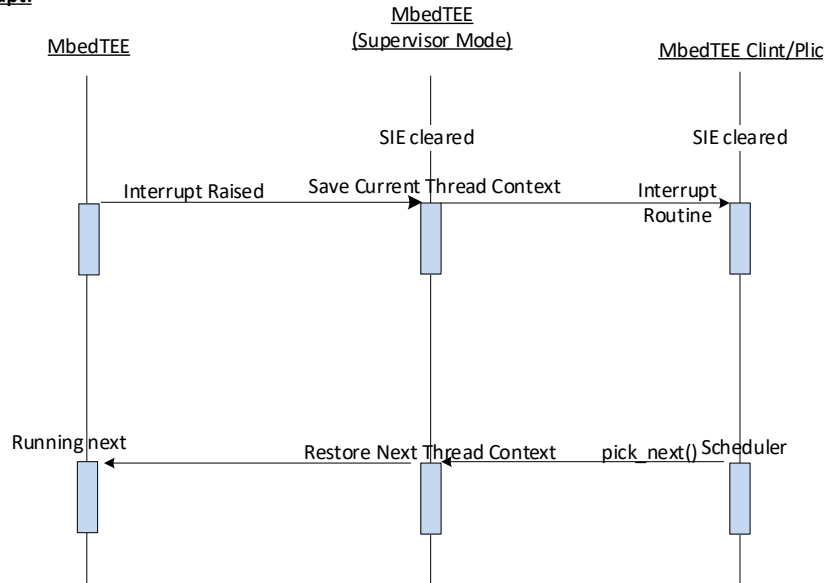


Figure 3-16 Interrupt – RISC-V

## 3.3 Context Switch of MIPS

Context switch may happen at any time during MIPS processor (CPU) execution, and may be triggered by the software interrupt or the external interrupt.

The following sections illustrate the different handling manners of different cases inside MbedTEE.

### 3.3.1 System Call

MIPS supports the SYSCALL instruction to make a request to the kernel mode, MbedTEE trusted application uses such software generated call to request the kernel service, which means the system call. The following figure illustrates the context switch on both the normal system call and scheduler system call.

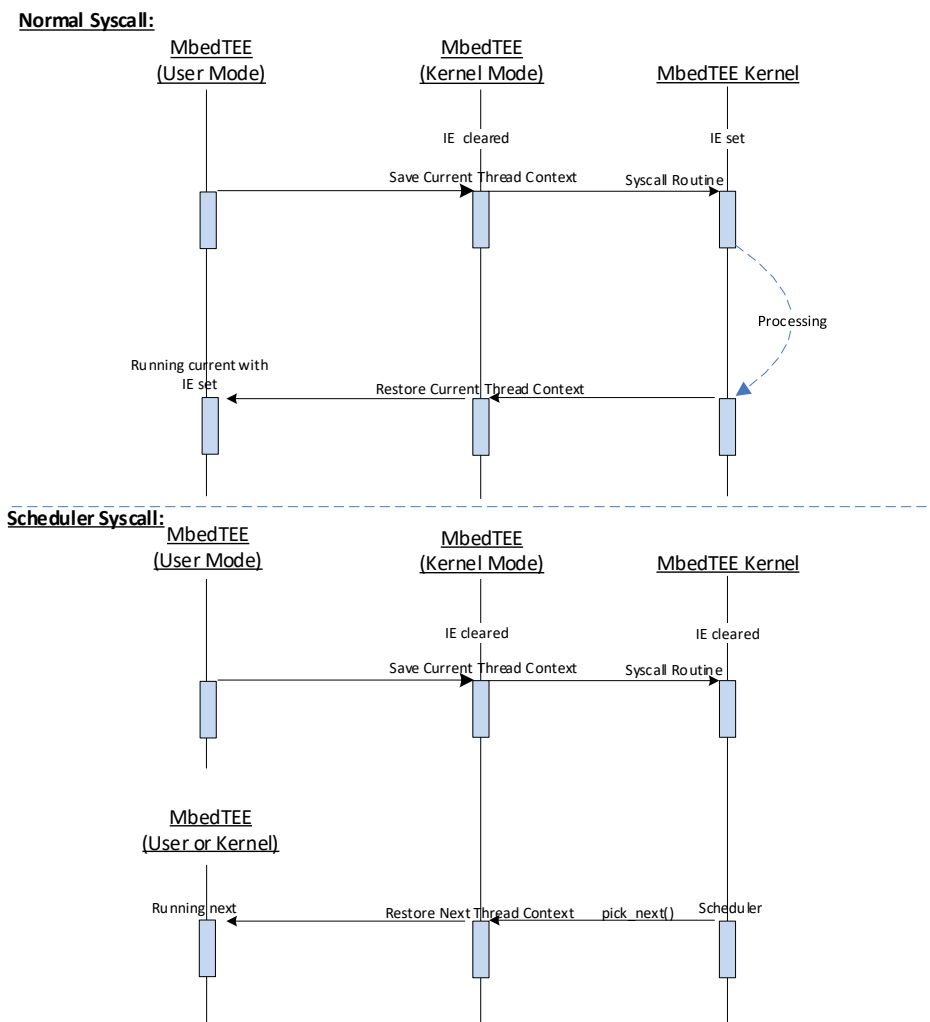


Figure 3-17 System Call - MIPS



## 3.3.2 Interrupt

Refer to the following figure to know the details of the MIPS normal interrupt and scheduler interrupt handler logics.

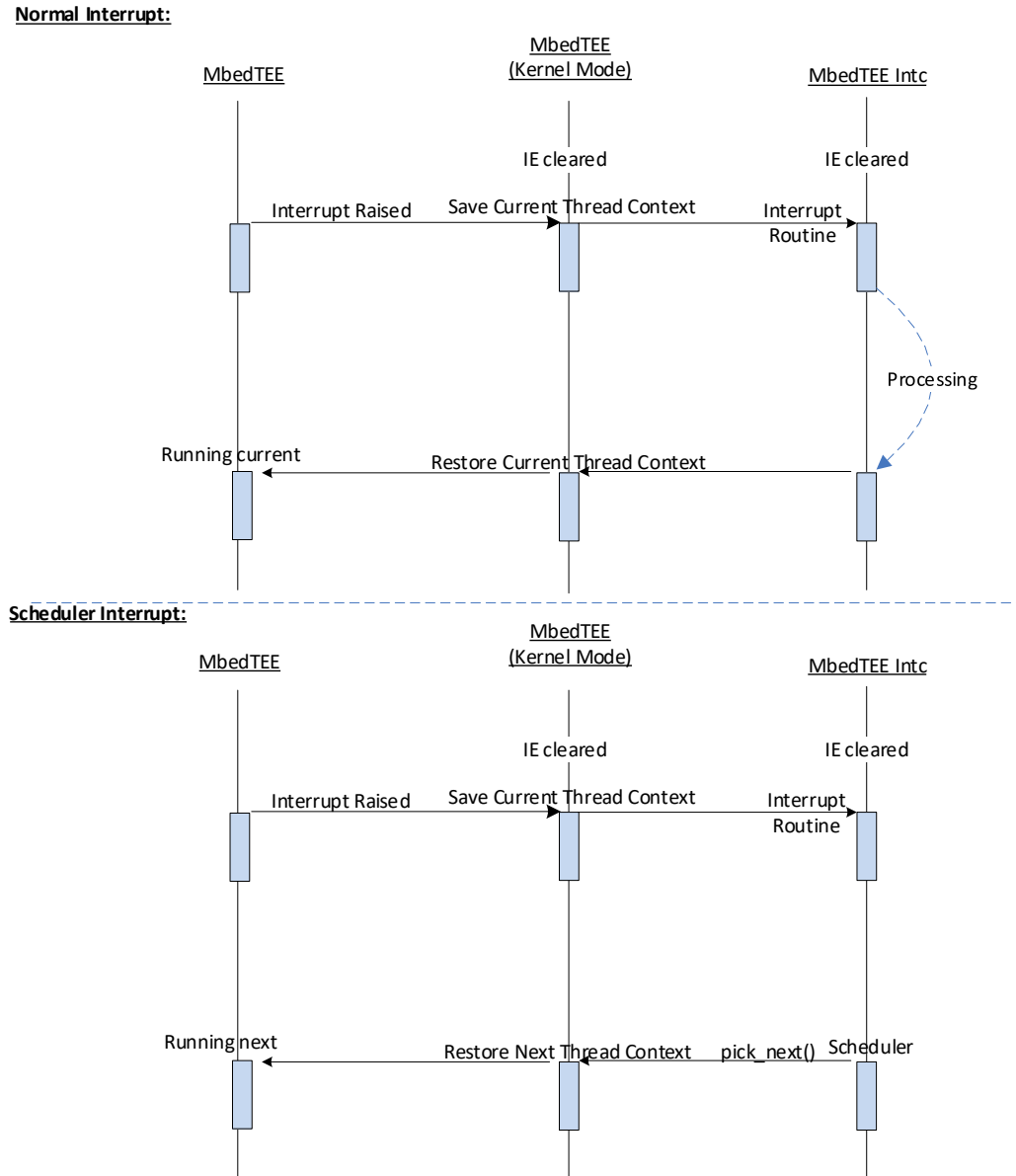


Figure 3-18 Interrupt - MIPS

# Chapter 4

## MbedTEE Components

## 4.1 Overview

MbedTEE kernel is consisted of the fundamental core and the kernel modules.

MbedTEE core provides the basic functionality of an operating system, such as the exception handling, scheduler, heap management, page management, timer framework, thread management, process management, system call, tasklets, workqueue, DTS, IPC, RPC, synchronization primitives and file system etc.

MbedTEE kernel module is a Linux-like module to extend the functionality of the core, it is usually used for describing a SoC peripheral driver. It is a piece of code that integrates into the kernel image. But now, it cannot be compiled separately into a dynamic loaded kernel object, so you cannot load or unload dynamically as the Linux module does.

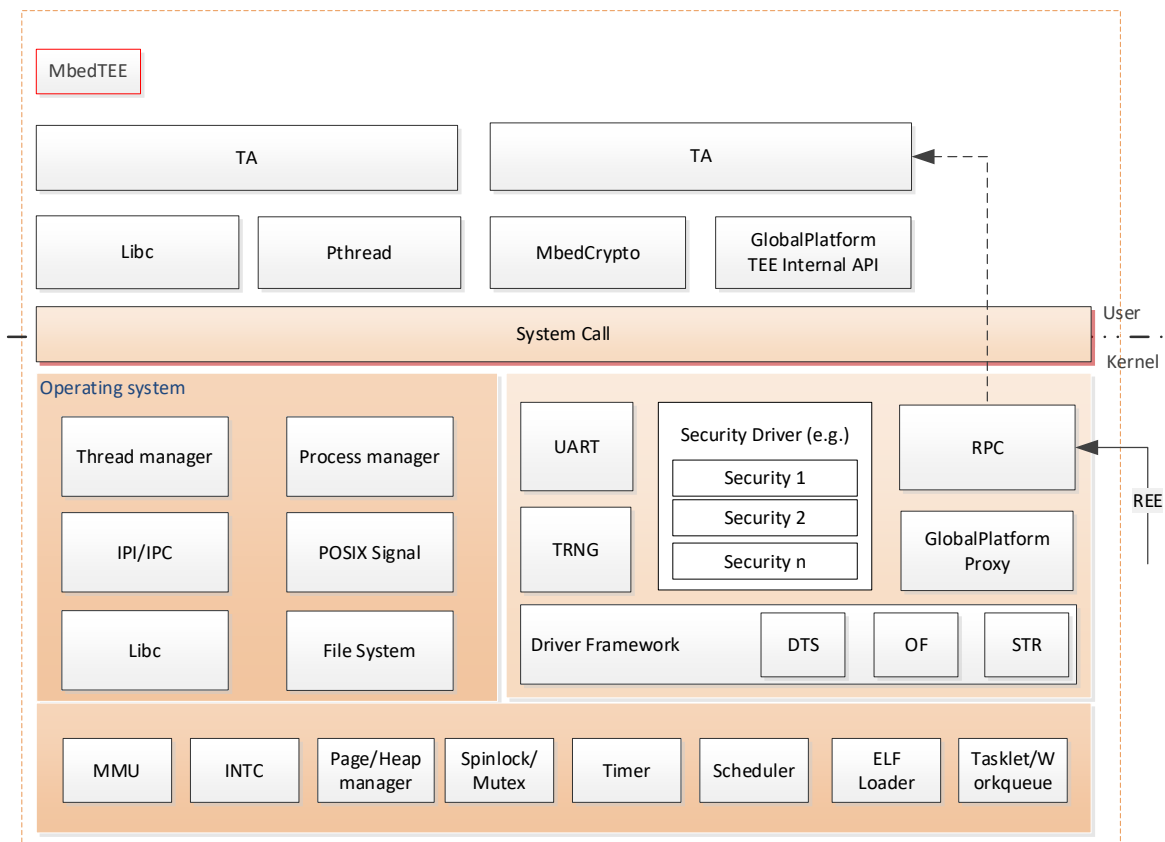


Figure 4-1 MbedTEE Components

The following sections introduce only the major components of the MbedTEE software.

## 4.2 System Call

MbedTEE provides the support of system call (syscall) for the TA to request the kernel services. In most of the operating systems, syscalls can only be made from the user space, MbedTEE is not different from these operating systems. System call is an exception that can be caught by the processor's exception handler which saves the calling thread's context and prepares the parameters for the destination function, then the handler jumps to the MbedTEE general syscall to process the destination function, the destination function will be executed with the calling thread's stack.

MbedTEE has a quite limited number of syscalls for security reasons, while many other modern operating systems have hundreds of syscalls. (e.g. the Linux has more than 300 syscalls).

Most of the MbedTEE syscalls are defined in the POSIX style, which allows TAs to be portable. The popular system calls like open, read, write, ioctl, close, lseek, mmap, stat, readdir, printf, poll, epoll and exit etc. are all supported.

MbedTEE has a C library (libc) in the user space to provide wrapper functions for the syscalls. The C library is built from the latest newlib with reentrant enabled and a retargeted lock for thread safety. The C library is released in a dynamic shared object (libc.so), MbedTEE kernel has ELF loader to load such DSOs.

## 4.3 Heap Manager

---

MbedTEE heap manager provides the service for other kernel components to allocate and free the kernel memory dynamically.

As we all know that dynamic use of memory pieces will cause memory fragmentation; in order to reduce memory fragmentation, MbedTEE supports multiple configurable heap allocator strategies. Three allocators are available:

- ✧ Slab allocator (default) - uses per-class freelists ( $O(1)$ ) for sub-page allocations up to 1024 bytes; larger requests are served directly by the page allocator.
- ✧ Slab + Buddy allocator - uses the Slab allocator for sub-page allocations backed by a buddy allocator for larger requests.
- ✧ Bitmap + Buddy allocator - uses bitmap tracking for small objects and a buddy allocator for larger requests.

MbedTEE heap manager provides support for:

- ✧ Allocate and free the physically continuous memory, the memory space is mapped with the continuous pages.
- ✧ Allocate and free the virtually continuous memory, the memory space is mapped with the scattered pages.

## 4.4 Page Manager

---

MbedTEE page manager provides the service for other kernel components to allocate and free the kernel pages dynamically.

In order to reduce page fragmentation, MbedTEE page allocation is under the control of the buddy algorithm.

MbedTEE page manager provides support for:

- ✧ Allocate and free the continuous pages.
- ✧ Allocate and free the scattered pages.

## 4.5 Timer Framework

---

MbedTEE uses CPU timer blocks (ARM generic timer, RISC-V sstc timer, RISC-V clint timer, XLNX xps timer on Microblaze-V, or MIPS CP0 count/compare timer) as the monotonic counter and source of timer events.

MbedTEE timer framework provides support for:

- ✧ Get the monotonic system time.
- ✧ Get/Set the system time.
- ✧ Create and destroy the timer event.
- ✧ Start and stop the timer event.
- ✧ Renew the timer event.
- ✧ Migrating the timer events when CPU hot-plug.

## 4.6 Synchronization Primitives

---

MbedTEE kernel provides the low-level mechanisms for the thread synchronization.

- ✧ atomic operation (32-bits)
- ✧ mutual exclusion (ARM 8-bits, others 32-bits)
- ✧ spinlock (ARM 8-bits, others 32-bits)
- ✧ semaphore (ARM 8-bits, others 32-bits)
- ✧ wait queue

When race conditions on multi-threaded shared resources are inevitable, synchronization primitives must be introduced to achieve thread safety; usually atomic operations, mutual exclusion, spinlocks, and semaphores can be used in such cases.

## 4.7 Tasklet

---

MbedTEE tasklet is a Linux-like mechanism for kernel modules to insert deferred work; it is typically used for handling the bottom halves of interrupts. taskletd runs with interrupts unmasked and has critical priority. With this mechanism, kernel modules can postpone long-running work to the taskletd, improving the latency of top-half interrupt handling.

If a tasklet routine is scheduled from an interrupt context, the taskletd will be executed immediately with its own stack after exiting the interrupt routine. If not, MbedTEE kernel raises the softint to execute taskletd ASAP. Linux softint borrows the current interrupted thread's stack when exiting IRQ boundary, unlike the Linux softint execution, MbedTEE uses the taskletd own stack (context) instead of the interrupted thread's stack.

## 4.8 Workqueue

---

MbedTEE workqueue is a mechanism for the kernel modules to insert deferred works, similar to the tasklet but with lower priority.

Workqueue supports two types of worker threads: Normal-Priority workers and High-Priority workers. Normal-Priority workers are designed for work that is not time-critical; High-Priority workers are designed for work that requires a strict response time.

Workqueue is designed to follow the policy "first come, first serve", but one workqueue may have several worker threads in parallel, meaning workqueue can only ensure the starting order of work items in the queue; it is unable to guarantee their completion order.

## 4.9 IPI

---

IPI is the inter processor interrupt and it is essentially the software generated interrupt which relied on the ARM-SGI or RISC-V-SSWI, the IPI driver is designed for the following purposes:. On ARM platforms, TEE-internal cross-CPU IPIs use SGI (IDs 0–15) local to the processor cluster. Note that TEE→REE RPC notification does NOT use SGI; instead it uses a dedicated GIC SPI (configured via the "rpc-t2r-spi" device-tree property) to signal the REE-side RPC handler.

- ✧ MbedTEE local cross-processor function calls (e.g. SMP TLB maintenance broadcasting)
- ✧ REE-TEE remote procedure calls (RPC)

## 4.10 RPC

---

The communication between REE and TEE relies on the MbedTEE RPC driver; RPC callee and caller both exist to provide bi-directional communication.

RPC driver is implemented based on the IPI (or ARM-SMC) and shared memory (REE memory). Due to the communication between different execution environments breaks the isolation boundary, all parameters passed via RPC must undergo rigorous security validation (see §5.7). The TEE→REE notify mechanism is platform-specific: on ARM platforms a dedicated GIC SPI (pend via `gic_pend_spi(rpc_t2r_spi)`) is used; on RISC-V platforms with IMSIC an MSI is raised (`im-sic_raise`); on RISC-V platforms without IMSIC a software interrupt (`softint_raise/SSWI`) is used.

Refer to section 5.7 for the detailed security checks.

## 4.11 IPC

---

MbedTEE provides POSIX signal, message queue, shared memory, pipe, eventfd, semaphore, process spawn/reaping, and user-space lock mechanisms for TA IPC and synchronization. The TA communication and resource sharing must go through explicit IPC. IPC establishment and usage are constrained by ACL/ownership policies (5.5) and kernel-side validation checks.

Security reference scope: section 5.6 details message queue session security and explicit file descriptor sharing (`mq_send_fd/mq_receive_fd`). Other IPC primitives in this chapter follow the same ACL gatekeeping model plus their corresponding subsystem checks.

### 4.11.1 Signal

---

MbedTEE supports POSIX-style signal primitives for TA process/thread control and asynchronous notification, including signal disposition/mask management, pending query, timed waiting, and suspension.

Interface coverage includes:

- `sigaction`, `sigprocmask`, `sigpending`
- `sigtimedwait`, `sigsuspend`, `pause`
- process-directed and thread-directed signal enqueue paths

Signal delivery path (high level):

- A sender enqueues signal metadata (siginfo) to process queue or thread queue according to target scope.
- Scheduler checks pending signals at safe preemption/return points and selects one deliverable signal according to mask and priority policy.
- For a caught signal, kernel builds the user signal-entry context and transfers control to user handler; after handler completion, sigreturn path restores the saved execution context.

Scheduler interaction:

- Signal handling is coordinated with scheduler state transitions, including wakeup of interruptible waits and stop/continue behavior (SIGSTOP/SIGCONT).
- Fatal/critical signals (SIGKILL, cancel/stop related paths) are prioritized over normal queued signals.

Security boundary:

- Kernel process is not signal-targetable by TA.
- Intra-process thread signaling is allowed; inter-process signaling is ACL/privilege constrained.
- Parent process and privileged process can signal target process; non-privileged cross-TA signaling requires IPC ACL permission.
- User-provided handler pointers and user buffers are validated before use.

---

## 4.11.2 Message Queue/Shared Memory/Pipe/Eventfd/Semaphore

---

Besides message queue and shared memory, MbedTEE IPC domain also includes pipe/eventfd/semaphore primitives for event notification and synchronization.

- Message queue (mq\_\*): command/data delivery between TAs with IPC ACL checks on session establishment and descriptor usage.
- Shared memory (shm\_\*): explicit data sharing via named objects, still bounded by process isolation and access checks.
- Pipe (pipe/pipe2): byte-stream IPC endpoint pair, poll/epoll-aware, suitable for producer-consumer data flow.
- Eventfd (eventfd/eventfd2): counter-based event source for lightweight wakeup/event aggregation, poll/epoll-aware.
- Semaphore (sem\_\*): classic synchronization primitive for cross-thread/process coordination in TA context.

Security constraints:

- IPC establishment and usage must pass ACL/ownership checks defined by TA configuration (ipc\_access, and dev\_access where applicable).
- Shared resources are always referenced by process-local descriptors; cross-TA sharing must use explicit kernel-mediated transfer path.
- Blocking IPC operations are interruptible by signal paths, avoiding unbounded uninterruptible waits in user-facing IPC APIs.

---

## 4.11.3 Process Spawn and Reaping

---

MbedTEE supports POSIX-style process creation and reaping for TA-to-TA process management.

Spawn (posix\_spawn):

- Creates a new TA process by name, loading the target ELF into a fresh address space.

- Supports `posix_spawn_file_actions_t` to configure the child's initial file descriptor table (open, close, dup2 actions applied before the child's entry point runs).
- Standard file descriptors (0/1/2) are inherited by default; other descriptors are closed unless explicitly included via file actions.
- The spawned child is registered as a waitable child of the calling process (`PROC_WAIT_WAITABLE`), enabling subsequent `waitpid` reaping.

Waitpid (`waitpid`):

- Waits for a specific child PID (`pid > 0`) or any child (`pid == -1`) to exit.
- Supports `WNOHANG` to poll child exit status without blocking.
- Only processes created via `posix_spawn` are waitable; the child's `_exit()` value is propagated to the caller's status argument.
- After reaping, the child process structure is released.

Security note: `spawn` path performs full TA ELF authentication (image security, 5.8) and ACL setup for the new process before execution begins.

## 4.11.4 User-Space Lock (`ulock`)

---

MbedTEE provides a kernel-backed fast-path for user-space pthread read/write locks via the `ulock` syscall interface. This avoids busy-wait in contended lock scenarios by suspending threads in the kernel until the lock becomes acquirable.

Interface:

- `wait_rdlock(ulock, timeout)` — suspends the calling thread until no writer holds the lock; increments the reader count atomically on success.
- `wait_wrlock(ulock, owner, timeout)` — suspends the calling thread until the lock is fully released; sets the exclusive writer flag atomically on success. The owner PID is used for priority inheritance: if a higher-priority thread is waiting for a write lock held by a lower-priority thread, `sched_inherit_prio()` temporarily boosts the lock holder's priority to prevent priority inversion.
- `wake_lock(ulock)` — wakes all threads waiting on the given `ulock` address after a lock release; used by the unlock path in user space.

The lock state is a 32-bit user-space atomic word shared between the kernel and user space. The kernel validates alignment and address accessibility (`access_ok`) before operating on it. All wait operations are interruptible by signals, with `EINTR` returned on interruption and `ETIMEDOUT` on expiry.

## 4.12 ELF Loader

---

MbedTEE ELF loader is used for loading ELF objects into each TA's virtual address space; it supports loading both executables and dynamic shared objects (DSOs).

ELF loader parses the TA ELF object, allocates scattered pages for the necessary segments, and finally maps these segments into the TA user space according to read-only/read-write/executable flags in the TA ELF LOADs. The mapping is done by the MMU driver with the TA private translation table.



The DSO is loaded in a similar way; the only difference is that there is no extra page allocation for the DSO's read-only segments. All TAs share the read-only pages of the DSO but with different user address mappings.

Address Space Layout Randomization (ASLR) is supported; the TA can be compiled and linked with the flag 'pie' and then ELF loader can load it to a random virtual address to enhance security. ELF loader also supports loading the executable TA that was not compiled or linked with the 'pie' flag.

The ELF loader always uses the ASLR - random virtual address for the DSO loading, thus the sources of the DSO must be compiled with the flag '-fPIC', otherwise the DSO relocation will be abnormal.

## 4.13 MMU

---

MbedTEE MMU driver focuses on the following functionalities:

- Cache operations
  - ✧ Cache clean
  - ✧ Cache invalidates
- Translation table management
- ASID management
- Memory mapping
  - ✧ Map the physical page to the kernel virtual address
  - ✧ Map the physical page to the user virtual address
  - ✧ Support the read-only, read-write, executable, non-cacheable and cacheable mapping flags
- Address translation
  - ✧ Kernel physical address to kernel virtual address translation
  - ✧ Kernel virtual address to kernel physical address translation
  - ✧ User virtual address to kernel physical address translation

## 4.14 File System

---

### 4.14.1 FS types

---

MbedTEE supports the following File systems:

- ✧ fatfs (RAM-based; also used as ramfs for transient TA storage)
- ✧ tmpfs
- ✧ devfs
- ✧ debugfs
- ✧ reefs (flash-based; AES-128-GCM block encryption, HKDF key derivation from HUK, anti-rollback via RPMB version counter)
- ✧ rpmbfs (RPMB-based; AES-128-GCM per-file encryption with per-file FEK, FAT encrypted with AES-128-GCM, HMAC-SHA256 RPMB MAC, hardware write-counter replay protection)

### 4.14.2 FS Operations

---

MbedTEE supports the following FS operations:

- ✧ open
- ✧ read
- ✧ write
- ✧ ioctl (only for device file)
- ✧ poll
- ✧ mmap
- ✧ close
- ✧ lseek
- ✧ fstat
- ✧ ftruncate
- ✧ rename
- ✧ unlink
- ✧ mkdir
- ✧ readdir
- ✧ rmdir

---

## 4.15 Storage

---

MbedTEE currently supports transient and persistent storage types; one is a volatile object while the other is a non-volatile object.

### 4.15.1 Transient

---

The transient object implementation is quite simple due to MbedTEE supporting tmpfs and ramfs (RAM-backed, with ramfs built on top of fatfs).

TA can leverage the ramfs operations mentioned in 4.14.2 to manage its own transient objects.

### 4.15.2 Persistent

---

MbedTEE supports two persistent storage backends, both providing confidentiality and integrity protection::

**reefs (REE File System backed):** Persistent files are encrypted and authenticated by MbedTEE, then forwarded to the REE's flash-based file system via RPC. REEFS derives root key/IV from HUK by HKDF, and uses AES-128-GCM for header/data protection. File UUID and block offset are bound as AAD to prevent cross-file or block relocation misuse. Decryption and validation are performed by MbedTEE before use. For anti-rollback, REEFS can mirror file headers to RPMB ('CONFIG\_REEFS\_ANTIROLLBACK\_RPMB'), then verify header version against RPMB baseline and reject rollbacked REE copies.

**rpmbfs (RPMB backed):** MbedTEE also supports a dedicated RPMB (Replay Protected Memory Block) secure file system. RPMBFS derives authentication/encryption materials from HUK by HKDF, uses per-file FEK with AES-128-GCM for data/FAT entry protection, and uses RPMB HMAC-SHA256 frame MAC plus hardware write-counter verification to detect replay/tampering. FAT/file version fields are incremented and verified as an additional integrity/consistency layer.

TA can leverage the corresponding fs operations mentioned in 4.14.2 to manage its own persistent objects.

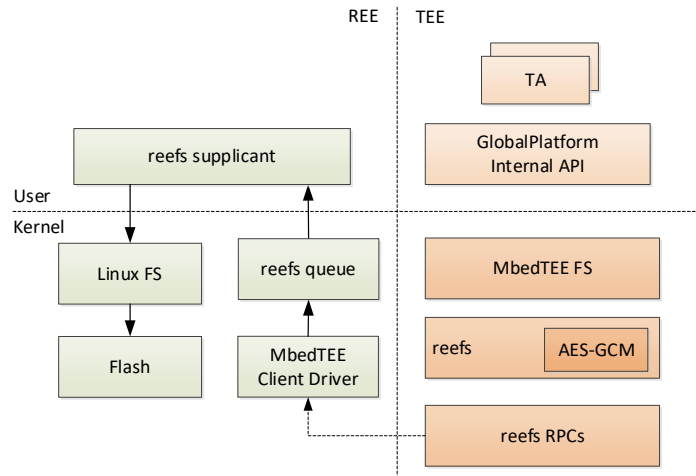


Figure 4-2 Persistent storage based on reefs

## 4.16DTB

MbedTEE supports the Flattened Device Tree (FDT) parser; there are two ways to specify the DTB for MbedTEE:

- ✧ Embedded DTB -> enabled by CONFIG\_EMBEDDED\_DTB, linked into MbedTEE kernel
- ✧ External DTB -> enabled by CONFIG\_DTB\_ADDR

## 4.17GlobalPlatform

MbedTEE supports the GlobalPlatform specified TEE working model and APIs.

- TEE Client API (mbedtee-client-api)
- TEE Client Driver (linux/drivers/tee/mbedtee)
- TEE Internal Core API (mbedtee-os/user/tee)

## 4.18MbedCrypto

MbedTEE embeds MbedCrypto as its internal cryptographic library, used by the TEE kernel, trusted services, and the TEE Internal Core API (GP TEE Core API) exposed to Trusted Applications. MbedCrypto is a self-contained, portable C library requiring no OS or floating-point support, making it well-suited for bare-metal TEE environments.

### 4.18.1 Typical In-Kernel Usage

The MbedCrypto library is used directly by multiple TEE subsystems:

- Secure storage (reefs/rpmbfs): AES-128-GCM for block-level authenticated encryption; HKDF (HMAC-SHA256) for per-file/per-FS key derivation from HUK; HMAC-SHA256 for RPMB frame MAC computation and verification.
- Image/TA security: AES-256-GCM, SM4-GCM, ChaCha20-Poly1305 for encrypted TA images; RSA-PSS, ECDSA (NIST/Brainpool), Ed25519 for image signature verification.

- GP TEE Internal Core API: Full algorithm set exposed to TAs via TEE\_CipherInit/Update/DoFinal, TEE\_DigestDoFinal, TEE\_MACInit/..., TEE\_AEInit/..., TEE\_AsymmetricSign/Verify/Encrypt/Decrypt, TEE\_DeriveKey, TEE\_GenerateRandom.

---

## 4.18.2 Symmetric Ciphers

---

AES (128/192/256-bit): ECB, CBC (NOPAD/PKCS5), CTR, CTS, XTS, GCM, CCM, SIV, CBC-MAC. AES-XTS uses 256-bit total (AES-128-XTS) or 512-bit total (AES-256-XTS) key pairs. AES-SIV (RFC 5297): nonce-misuse-resistant AEAD.

SM4 (128-bit): ECB (NOPAD/PKCS5), CBC (NOPAD/PKCS5), CTR, GCM, CCM. ChaCha20 (256-bit): stream mode and ChaCha20-Poly1305 AEAD (RFC 8439). DES (64-bit) and 3DES (128/192-bit): ECB, CBC (NOPAD/PKCS5), CBC-MAC.

---

## 4.18.3 AEAD (Authenticated Encryption with Associated Data)

---

AES-GCM (4–16-byte tag), AES-CCM (4–16-byte even tag), SM4-GCM, SM4-CCM, ChaCha20-Poly1305 (16-byte tag fixed). Streaming AEAD API: setkey / start / update\_aad / update / final.

---

## 4.18.4 Hash / Message Digest

---

MD5 (128-bit), SHA-1 (160-bit), SHA-224/256/384/512 (SHA-2 family), SHA3-224/256/384/512 and SHAKE256-XOF (SHA-3/Keccak), SHA-256/192 (truncated SHA-256 variant), SM3 (256-bit, GM/T 0004).

---

## 4.18.5 Message Authentication Code (MAC)

---

HMAC: HMAC-MD5, HMAC-SHA1, HMAC-SHA224/256/384/512, HMAC-SHA3-224/256/384/512, HMAC-SM3. CMAC (AES, NIST SP 800-38B). CBC-MAC (AES/DES/3DES, NOPAD and PKCS5 variants). Poly1305 (standalone and as part of ChaCha20-Poly1305).

---

## 4.18.6 Key Derivation Functions (KDF)

---

HKDF (HMAC-SHA256, RFC 5869): extract / expand / derive / derive\_hash variants; used internally for HUK-derived key hierarchy. PBKDF2 (RFC 2898): configurable hash. Both accessible via GP API (TEE\_ALG\_HKDF).

---

## 4.18.7 Asymmetric Key Exchange and Encryption

---

RSA encrypt/decrypt (512–4096-bit): Raw, PKCS#1 v1.5, OAEP-MGF1 with SHA1/224/256/384/512 and SHA3-224/256/384/512. DH key agreement. ECDH over NIST P-192/256/384/521 and Brainpool BP256/384/512R1. X25519 (RFC 7748), X448 (RFC 7748). SM2-PKE (GM/T 0003). SM2-KEP (GM/T 0003).

---

## 4.18.8 Digital Signatures

---

RSA PKCS#1 v1.5 sign/verify: MD5, SHA1, SHA224/256/384/512, SHA3-224/256/384/512. RSA-PSS (MGF1): SHA1, SHA224/256/384/512, SHA3-224/256/384/512. ECDSA over NIST P-192/256/384/521 and Brainpool BP256/384/512R1:

SHA1/224/256/384/512, SHA3-224/256/384/512. DSA (FIPS 186): SHA1, SHA224/256, SHA3-224/256/384/512. Ed25519 (RFC 8032), Ed448 (RFC 8032). SM2-DSA with SM3 (GM/T 0003).

## 4.18.9 Utilities / Big-Number

---

Bignum/MPI arithmetic (basis for all public-key operations). ASN.1/DER encoder-decoder (RSA/ECDSA/DSA keys and signatures). Base64 encode/decode. ECP point arithmetic for Short-Weierstrass and Montgomery curves.

All MbedCrypto primitives operate entirely within TEE secure memory; no secret material crosses the secure-world boundary. Algorithm availability to TAs is gated by the GP TEE algorithm dispatch table in `user/tee/tee_crypto.c` and build-time Kconfig selection.

# Chapter 5

## TEE Security

## 5.1 Isolation

---

This section focuses on the REE-TEE world isolation and User-Kernel space isolation.

### 5.1.1 REE-TEE Isolation

---

TEE should be isolated from REE by hardware-enforced security mechanisms. For ARM TrustZone systems, isolation is primarily enforced by Secure/Non-secure world separation. For RISC-V and other non-TrustZone systems, isolation is enforced by SoC security partitioning (for example dedicated cores, memory regions, and peripheral access control).

REE and TEE should each have dedicated physical memory and dedicated hardware IPs in the SoC. The SoC architecture/bus design should be able to detect whether an access is from the REE or TEE state, so the IPs can authorize different accesses from different states. The detailed memory isolation and IPs isolation for REE and TEE are out of scope of current document, each should be reviewed according to the features of the target product SoC.

From the ARM TrustZone view, in the same processor the isolations are:

- ✧ Each has dedicated MMU.
- ✧ Each cache line has NS (non-secure) flag, the REE cache operations can't evict the TEE cache lines.
- ✧ Each has dedicated timer (only frequency configuration is shared), only TEE can configure the timer frequency.
- ✧ TEE has higher permission in the interrupt controller, TEE determines which interrupt will be routed to TEE and which interrupt can be routed to REE.
- ✧ For AArch32, each has dedicated interrupt mode, TEE uses the FIQ while REE uses the IRQ. REE is unable to change the FIQ configuration. For AArch64, each has dedicated IRQ.
- ✧ Monitor runs in the secure state only.
- ✧ TEE can enable or disable the REE access to some processor functionalities. (e.g. TEE can enable or disable REE access to the floating-point extension).
- ✧ TEE instruction fetches from the REE memory is not permitted.

Since TEE is isolated from REE, a bridge must be present when REE needs to access TEE services. The secure monitor acts as the bridge in the ARM TrustZone based architecture.

For ARM TrustZone based systems, the bridge hardening rules include:

- ✧ Each processor can disable the SMC instruction from REE.
- ✧ Each parameter from the REE is checked to prevent malicious software attacks through mock parameters.
- ✧ Each parameter is backed up in TEE memory before the check is performed; this prevents TOCTOU (Time of Check / Time of Use) attacks.

### 5.1.2 User-Kernel Isolation

---

MMU is quite a mandatory module for the security, especially for the isolation of the user-kernel address spaces and the isolation of TA process space.

## 5.1.2.1 Space Isolation

On ARM TrustZone systems, REE and TEE run on the same processor with banked secure/non-secure MMU context, so each world has dedicated MMU configurations. On RISCv/non-TrustZone systems, REE and TEE may run on different dedicated cores, but each still maintains isolated MMU mappings and user/kernel address-space separation per core. (Configurations are related to translation table base control, translation table base addresses, user and kernel address-space separation, domain separation, application address-space separation, and i-cache/d-cache working mode.)

Parameter checks are applied:

- ✧ With this separation, the address passed from user space can be validated easily (pointers from the system calls). E.g. MbedTEE kernel provides the API `copy_from_user` and `copy_to_user` for the kernel modules to exchange data with the user space, these APIs validate the user pointer and size by easily check if they are within predefined user space range. All the kernel modules which may involve the user space address must go through these two APIs.
- ✧ These APIs also check whether the user space address has the valid entry in the calling TA's translation table (valid mapping and proper access permission).

### 5.1.2.1.1 AArch32 Space Isolation

User-kernel address space separation: AArch32 MMU supports two translation tables, one is used for the user space translation table walk, another one is used for the kernel space translation table walk. Each translation table has dedicated address space and there is no overlap of these two address spaces. According to the hard-coded configuration of the AArch32 MMU driver:

- ✧ TTBR0 for User 0x00000000 ~ 0x3FFFFFFF (1GB)
- ✧ TTBR1 for kernel 0x40000000 ~ 0xFFFFFFFF (3GB)

AArch32 user space is unable to touch the TTBR1 range directly to injure the kernel space, AArch32 MMU driver enforces the access permission of kernel space to “User no access” if it detects that the input translation table is for the kernel.

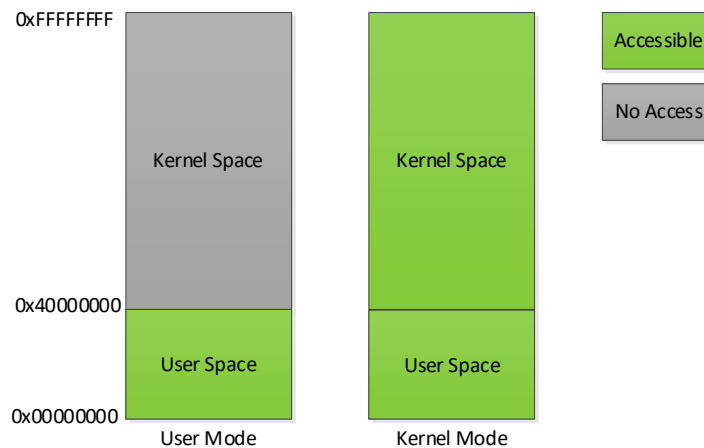


Figure 5-1 AArch32 Address Space



### 5.1.2.1.2 AArch64 Space Isolation

User-kernel address space separation: AArch64 MMU supports two translation tables, one is used for the user space translation table walk, another one is used for the kernel space translation table walk. Each translation table has dedicated address space and there is no overlap of these two address spaces. According to the hard-coded configuration of the AArch64 MMU driver:

- ✧ TTBR0 for User 0x0000000000000000 ~ 0x0000007FFFFFFFFF (512GB)
- ✧ TTBR1 for kernel 0xFFFFFFFF8000000000 ~ 0xFFFFFFFFFFFFFFFF (512GB)

AArch64 user space is unable to touch the TTBR1 range directly to injure the kernel space, AArch64 MMU driver enforces the access permission of kernel space to “User no access” if it detects that the input translation table is for the kernel.

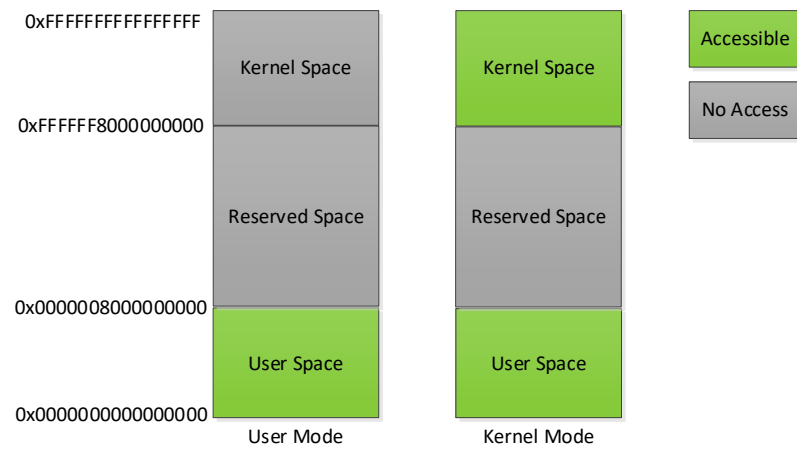


Figure 5-2 AArch64 Address Space

### 5.1.2.1.3 RISCv32 Space Isolation

According to the hard-coded configuration of the RISCv32 SV32 MMU driver:

- ✧ User 0x00000000 ~ 0x7FFFFFFF (2GB)
- ✧ Kernel 0x80000000 ~ 0xFFFFFFFF (2GB)

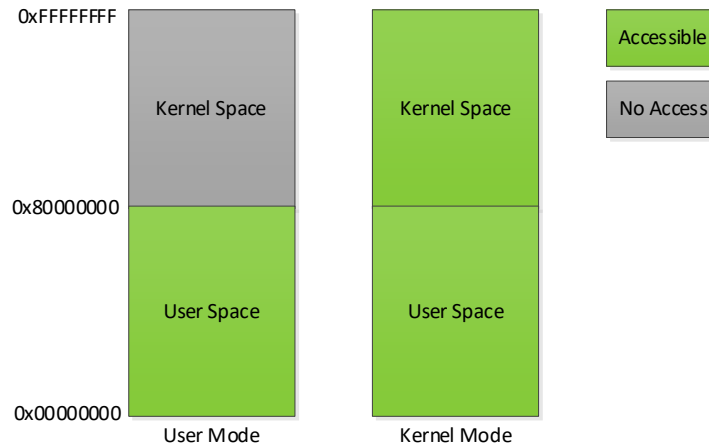


Figure 5-3 RISC32 SV32 Address Space

#### 5.1.2.1.4 RISC32 Space Isolation

According to the hard-coded configuration of the RISC32 SV32 MMU driver:

- ✧ User 0x0000000000000000 ~ 0x0000003FFFFFFFFF (256GB)
- ✧ Kernel 0xFFFFFFFF00000000 ~ 0xFFFFFFFFFFFFFFFF (256GB)

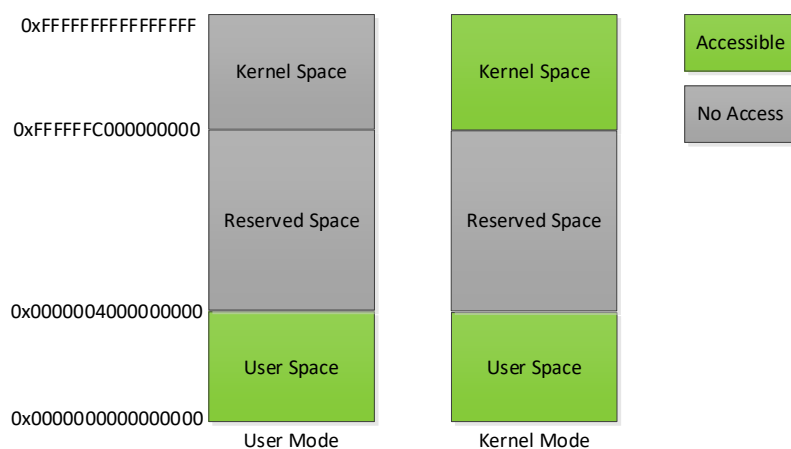


Figure 5-4 RISC32 SV39 Address Space

#### 5.1.2.1.5 MIPS32 Space Isolation

According to the hard-coded configuration of the MIPS32 MMU driver:

- ✧ User 0x00000000 ~ 0x7FFFFFFF (2GB)
- ✧ Kernel 0x80000000 ~ 0xFFFFFFFF (2GB)

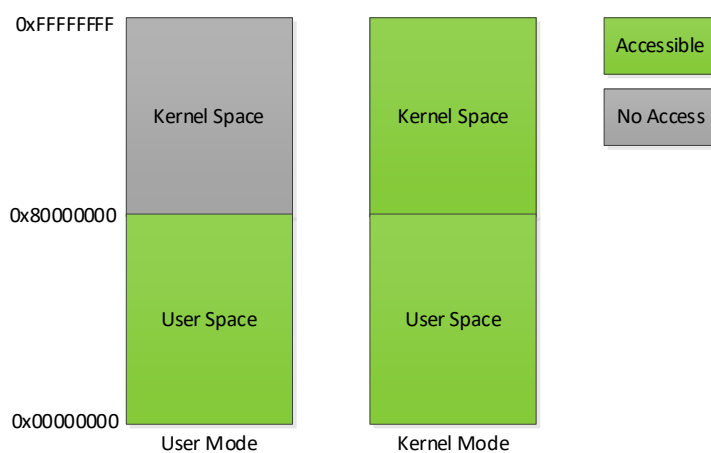


Figure 5-5 MIPS32 Address Space

### 5.1.2.2 Execution Isolation

---

According to the section 2.3, the processor provides multiple exception levels, with this hardware separation the user and kernel executions can be explicitly isolated.

- ✧ An exception can never be taken into a lower exception level.
- ✧ An exception return can never be to a higher exception level.

### 5.1.2.3 Resource Isolation

---

According to the previous sections, the processor provides non-privileged and privileged modes. With this hardware enforcement the user and kernel access to the system resources is explicitly separated.

- ✧ Each processor mode has its dedicated stack.
- ✧ Each processor mode has dedicated registers, except the generic registers are shared with each other. The software exception routines handle the registers properly.
- ✧ Non-privileged mode (user) is denied to configure certain system configurations, e.g. MMU configuration, interrupt configuration, monitor configuration, cache configuration and SoC registers etc.
- ✧ Non-privileged mode (user) must delegate to the system calls to access the system resources.

### 5.1.3 TA Isolation

---

- Memory Isolation between TAs:
  - ✧ Each TA has its own translation table.
  - ✧ Each TA has its own address space.
  - ✧ According to the ELF Loader, the pages for storing the TA ELF object are dynamically allocated. TAs can only share the read-only pages of the DSO.
  - ✧ Each TA thread has its own stack, the stack pages are dynamically allocated.
  - ✧ Each TA has its own heap, the heap pages are dynamically allocated.
  - ✧ ASLR applied to each TA.
- Resource Isolation between TAs:
  - ✧ The opened object handlers (e.g. file descriptors) are only visible within the owner TA, and they have no meaning in other TA.
  - ✧ The resources under the handlers are always dynamically allocated for the multi-session supported modules.
  - ✧ The resource sharing between TAs must go through the explicit IPC.
  - ✧ Crashes in one TA cannot stop other TAs or the kernel's execution. Crashes in one TA are always recoverable, requiring only a restart of the TA rather than the entire system.

## 5.2 ELF Mapping

---

MbedTEE kernel has multiple sections in the ELF binary, and each section has its own MMU mapping flag. With these flags, MbedTEE kernel can easily protect the “.text” and “.rodata” sections from software modification by the processor, but it still can’t prevent the malicious overwrite from the hardware DMA.

**Table 5-1 Kernel ELF Segments**

| Item    | RW/RO | Exec/non-exec  |
|---------|-------|----------------|
| .text   | RO    | Executable     |
| .rodata | RO    | Non-Executable |
| .init   | RO    | Executable     |
| .data   | RW    | Non-Executable |
| .ramfs  | RW    | Non-Executable |
| .bss    | RW    | Non-Executable |

The TA also has multiple ELF sections and each section has its own MMU mapping too. The TA memory is dynamically allocated by MbedTEE kernel during TA loading, and TA memory segments will be mapped to the virtual user space with the associated flags.

According to the ELF LOAD flags in the program header, the TA is always mapped as follow. Generally, each TA has two LOAD segments, one is for the read-only and executable sections, the other one is for the read-write and non-executable sections. Note that .rodata shares the same PT\_LOAD segment as .text on current TA layout, and both are mapped as RX (read+execute) when that segment carries PF\_R | PF\_X.

**Table 5-2 TA ELF Segments**

| Item    | RW/RO | Exec/non-exec |
|---------|-------|---------------|
| .hash   | RO    | Executable    |
| .dynsym | RO    | Executable    |
| .dynstr | RO    | Executable    |

|          |    |                |
|----------|----|----------------|
| .rel.dyn | RO | Executable     |
| .rel.plt | RO | Executable     |
| .text    | RO | Executable     |
| .rodata  | RO | Executable     |
| .dynamic | RW | Non-Executable |
| .got     | RW | Non-Executable |
| .data    | RW | Non-Executable |
| .bss     | RW | Non-Executable |

**Note:** The ELF loader maps segments according to the `PT_LOAD` program header flags. Typically `.hash`, `.dynsym`, `.dynstr`, `.rel.dyn`, `.rel.plt`, `.text`, and `.rodata` all reside in the same read-only `PT_LOAD` segment with `PF_R | PF_X` flags, so they are all mapped as executable. Splitting `.rodata` into a dedicated non-executable `PT_LOAD` segment would further reduce the executable memory surface and is a potential future enhancement.

## 5.3 Timer

---

### 5.3.1 Overview

---

This chapter first briefly describes the hardware mechanism provided to guarantee a secure timer, and then introduces how time is managed in MbedTEE.

### 5.3.2 Monotonic Counter

---

MbedTEE has a dedicated timer (ARM generic timer, RISC-V sstc timer, RISC-V clint timer, Xilinx xps timer on Microblaze-V, or MIPS CP0 count/compare timer) which is isolated with the REE. This timer provides the monotonic counter and system timer events with interrupt support.

Once started, the monotonic counter will ONLY increase. It will not decrease or stop as long as power is supplied. It guarantees the time is monotonic from HW.

### 5.3.3 Time Category

---

MbedTEE supports Global Platform Time APIs. The API provides access to three sources of time:

- System Time
  - ✧ The System Time is also known as TEE time, which is calculated from the Monotonic Counter. The System time is not required to be equal to the real time of the real world, but it's monotonic after the system boots.
- TA Time
  - ✧ The GlobalPlatform `TEE_GetTAPersistentTime` function retrieves the persistent time of the TA expressed as a number of seconds and milliseconds since the arbitrary origin set by calling `TEE_GetTAPersistentTime`, the persistent time of each TA is so-called TA Time.
  - ✧ TA Time has the following features:
    - ⌘ TA Time source is not from HW Timer, but from corresponding TA.
    - ⌘ TA Time can be different from System time.
    - ⌘ One TA Time can be different from another TA Time.
    - ⌘ One TA only supports one persistent time at any moment.
    - ⌘ TA Time is set / updated by TA through GP API, but this time is also required monotonic.
- REE Time
  - ✧ The REE Time is time retrieved from REE side. In normal case, the REE time source is from REE timer. The timer is not required to be secure. Therefore, the REE Time should not be considered as trusted, as it may be tampered by the user or the REE software.

In GlobalPlatform time API, when a TA sets the TA persistent time for the first time, the kernel records the time offset between the System Time and such TA Time. When a TA gets the TA persistent time after some moment, GlobalPlatform time API gets the current System Time, gets the time offset, calculates the current TA time, and returns it to the TA.

To keep the TA time monotonic, when a TA updates an existing TA persistent time, GlobalPlatform time API will check whether the new TA persistent time is greater than the current TA persistent time. If the new TA time is not greater than the current TA time, the request will be rejected.

## 5.4 Debugging

- To ensure the security, the following mechanisms can only be enabled at development stage:
  - ✧ User backtrace
  - ✧ Kernel backtrace
  - ✧ User TA print to the serial terminal
  - ✧ Kernel prints the critical message (e.g. TA abort reason or kernel panic/oops messages)
  - ✧ Ps supported with necessary thread information

```

ps
TID|PID|CPU  STATE  SG|UC|KC|E|B|A|PRI|POLICY  RUNTIME  LOADING  NAME
0001|0000@0  Running 00|00|01|0|1|1|00|00@RR  000008543.191893877s  099.99% 100.00% idle@0001
0002|0000@0  waiting 00|00|01|0|1|1|62|62@FIFO  000000000.000000583s  000.00% 000.00% taskletd@0002
0004|0000@0  waiting 00|00|01|0|1|1|44|44@RR  000000000.042801068s  000.00% 000.00% kworker@0004
0005|0000@0  waiting 00|00|01|0|1|1|48|48@RR  000000000.032266401s  000.00% 000.00% kworker-H@0005
0006|0000@6  Running 00|00|01|0|0|ff|62|62@FIFO  000000000.012955621s  000.00% 000.00% kshell@0006
0008|0000@1  Running 00|00|01|0|1|2|00|00@RR  000008543.374449852s  099.99% 100.00% idle@0008
0009|0000@1  waiting 00|00|01|0|1|2|62|62@FIFO  000000000.000000543s  000.00% 000.00% taskletd@0009
0011|0000@1  waiting 00|00|01|0|1|2|44|44@RR  000000000.035874755s  000.00% 000.00% kworker@0011
0012|0000@1  waiting 00|00|01|0|1|2|48|48@RR  000000000.056042454s  000.00% 000.00% kworker-H@0012
0014|0000@2  Running 00|00|01|0|1|4|00|00@RR  000008542.063547270s  099.99% 100.00% idle@0014
0015|0000@2  waiting 00|00|01|0|1|4|62|62@FIFO  000000000.000000545s  000.00% 000.00% taskletd@0015
0017|0000@2  waiting 00|00|01|0|1|4|44|44@RR  000000000.034708845s  000.00% 000.00% kworker@0017
0018|0000@2  waiting 00|00|01|0|1|4|48|48@RR  000000000.044951762s  000.00% 000.00% kworker-H@0018
0020|0000@3  Running 00|00|01|0|1|8|00|00@RR  000008543.379188196s  099.99% 100.00% idle@0020
0021|0000@3  waiting 00|00|01|0|1|8|62|62@FIFO  000000000.000000589s  000.00% 000.00% taskletd@0021
0023|0000@3  waiting 00|00|01|0|1|8|44|44@RR  000000000.034563784s  000.00% 000.00% kworker@0023
0024|0000@3  waiting 00|00|01|0|1|8|48|48@RR  000000000.030651331s  000.00% 000.00% kworker-H@0024
0026|0000@4  Running 00|00|01|0|1|10|00|00@RR  000008542.298252042s  099.99% 100.00% idle@0026
0027|0000@4  waiting 00|00|01|0|1|10|62|62@FIFO  000000000.000000589s  000.00% 000.00% taskletd@0027
0029|0000@4  waiting 00|00|01|0|1|10|44|44@RR  000000000.034611118s  000.00% 000.00% kworker@0029
0030|0000@4  waiting 00|00|01|0|1|10|48|48@RR  000000000.028300859s  000.00% 000.00% kworker-H@0030
0032|0000@5  Running 00|00|01|0|1|20|00|00@RR  000008542.239626476s  099.99% 100.00% idle@0032
0033|0000@5  waiting 00|00|01|0|1|20|62|62@FIFO  000000000.000000589s  000.00% 000.00% taskletd@0033
0035|0000@5  waiting 00|00|01|0|1|20|44|44@RR  000000000.035189466s  000.00% 000.00% kworker@0035
0036|0000@5  waiting 00|00|01|0|1|20|48|48@RR  000000000.055133759s  000.00% 000.00% kworker-H@0036
0038|0000@6  Ready   00|00|01|0|1|40|00|00@RR  000008542.165304066s  099.99% 099.99% idle@0038
0039|0000@6  waiting 00|00|01|0|1|40|62|62@FIFO  000000000.000000589s  000.00% 000.00% taskletd@0039
0041|0000@6  waiting 00|00|01|0|1|40|44|44@RR  000000000.034628510s  000.00% 000.00% kworker@0041
0042|0000@6  waiting 00|00|01|0|1|40|48|48@RR  000000000.053340790s  000.00% 000.00% kworker-H@0042
0044|0000@7  Running 00|00|01|0|1|80|00|00@RR  000008542.520373297s  099.99% 100.00% idle@0044
0045|0000@7  waiting 00|00|01|0|1|80|62|62@FIFO  000000000.000000589s  000.00% 000.00% taskletd@0045
0047|0000@7  waiting 00|00|01|0|1|80|44|44@RR  000000000.036087821s  000.00% 000.00% kworker@0047
0048|0000@7  waiting 00|00|01|0|1|80|48|48@RR  000000000.053136373s  000.00% 000.00% kworker-H@0048
Current: 33/8192 | 0000 Suspend | 0000 Exiting | 0024 Waiting | 0000 Sleep | 0001 Ready | 0008 Running
cpuidle: CPU0=100.00% CPU1=100.00% CPU2=100.00% CPU3=100.00% CPU4=100.00% CPU5=100.00% CPU6=99.99% CPU7=100.00%
ReadyCnt: CPU0=[0/4] CPU1=[0/4] CPU2=[0/4] CPU3=[0/4] CPU4=[0/4] CPU5=[0/4] CPU6=[1/5] CPU7=[0/4]
#

```

Figure 5-6 PS information

## 5.5 Access Control Policy

---

Each TA has one ACL (Access Control List) alongside the TA configuration. Example:

```
name = "shell";
uuid = "eb2f9996-9430-4905-a4fc-8476736e0d7b";
path = "/apps/shell.elf";
version = "1.0.1";
stack_size = "8192";
heap_size = "1048576";
single_instance = "1";
multi_session = "0";
inst_keepalive = "0";
dev_access = "/dev/uart0, /dev/urandom";
ipc_access = "xxx";
```

The ACL contains two access control policies:

- Device Access Control, which prevents unauthorized access to devices by the TA. The TA is only authorized to access the devices specified in the **dev\_access** list.
- IPC Access Control, which prevents unauthorized IPC with the peer TA. The TA is only authorized to have the IPC with the TAs specified in the **ipc\_access** list.

## 5.6 IPC Security

---

This section focuses on message queue session establishment and explicit file descriptor sharing security checks. For other IPC primitives (signal/shared memory/pipe/eventfd/semaphore/ulock), refer to section 4.11 for behavior and section 5.5 for ACL policy baseline.

### 5.6.1 Message Queue

---

MbedTEE provides standard POSIX message queues for sharing messages between TAs.

According to the section 5.5, MbedTEE kernel enforces the check on the IPC between TAs. MbedTEE kernel records the *name* of each **mq\_open** to the kernel IPC session list along with the owner's information. There are multiple processing logics on the establishment:

- if the name does not exist in the IPC session list, kernel will create a new IPC session (**O\_CREAT** will also be considered) and record this IPC name, new generated IPC descriptor and the calling TA's name/PID to this session, the IPC descriptor will also be recorded to the calling TA's kernel structure (process structure in kernel).
- if the name already exists in the IPC session list, then kernel checks if the calling TA has the right to establish the IPC on this session, the session owner's name must be present in the calling TA's **ipc\_access** list. If granted, the calling TA's name/PID will be recorded to the session structure, also the IPC descriptor will be recorded to the calling TA's kernel structure (process structure in kernel). The reference counter of this session will be increased by 1.

When using the send, receive or other functions, the kernel performs the checks on the IPC descriptor, the descriptor must be present in the calling TA's kernel structure; otherwise, a permission denial will be returned.



## 5.6.2 File descriptor sharing

---

In order to allow explicit file descriptor sharing between TA, the POSIX message queues have been extended with the following functions, where `fd` is the file descriptor to be shared:

```
mq_send_fd(mq_descriptor, fd)
mq_receive_fd(mq_descriptor, &fd)
```

These functions have the following security checks in the kernel:

- **ipc\_access** check which was already introduced in above section
- **dev\_access** check is also involved, the receiver must have the access right to the FD associated device.

When access is granted, the FD will be duplicated on the same resource to the receiver's process space, and thus the reference counter of this resource will be increased by 1. The MbedTEE kernel releases the resource when both TA close their file descriptor.

## 5.7 RPC Security

---

- All the RPC parameters should be checked.
- All the RPC pointers and their sizes should be checked to ensure TEE memory/IPs are not involved, MbedTEE kernel provides an API to check if the input pointer/size is within, overlaps, or totally covers the TEE memory.
- All the RPC structures should have backup in TEE memory, MbedTEE uses the backup in TEE to check the parameter to avoid the TOCTOU.

## 5.8 Image Security

---

TA images and certificates in MbedTEE are encrypted and/or signed by the build-time signing tool and verified by the `auth_ta` (authentication Trusted Application) at load time. The following cryptographic algorithms are currently in use for the image protection:

### Encryption:

- AES-256-GCM
- SM4-GCM
- ChaCha20-Poly1305

### Signing:

- RSA-PSS with SHA-256/SHA-384/SHA-512 (key sizes: 2048, 3072, 4096 bits)
- ECDSA NIST curves: P-256/SHA-256, P-384/SHA-384, P-521/SHA-512
- ECDSA Brainpool curves: BP-256/SHA-256, BP-384/SHA-384, BP-512/SHA-512
- Ed25519

*Thanks for reading*